OptCCL: Scalable Synthesis of Optimal Collective Communication Algorithms

Richard Shapley
Cornell University

Rachit Agarwal
Cornell University

David Shmoys
Cornell University

Abstract

We present OptCCL, a technique to synthesize collective communication algorithms that are optimal for a given host and network hardware and topology. OptCCL is general: it enables synthesizing optimal algorithms for all existing collectives, for all existing hardware, and even for multiple concurrent collectives sharing host and network resources. And yet, OptCCL is scalable: it synthesizes optimal algorithms for hundreds of GPUs within tens of minutes.

CCS Concepts

• **Networks → Network algorithms**; **Data center networks**; • **Theory of computation → Design and analysis of algorithms**; • **Computing methodologies → Machine learning**.

Keywords

Machine Learning, Collective Communication, Scheduling

ACM Reference Format:
Richard Shapley, Rachit Agarwal, and David Shmoys. 2026. OptCCL: Scalable Synthesis of Optimal Collective Communication Algorithms. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26), August 17–21, 2026, Denver, CO, USA.* ACM, New York, NY, USA, 25 pages. https://doi.org/10.1145/3789240.3829207

1 Introduction

Collective communication in machine learning (ML) training and inference refers to inter-GPU communication for synchronizing gradients, parameters, and optimizer states. Recent studies have established that performance of collective communication algorithms can significantly impact end-to-end performance of ML (training and inference) jobs as well as GPU resource utilization [4, 11, 20, 27].

Designing optimal collective communication algorithms (defined formally in §2, but intuitively, one that minimizes the time taken to complete the collective) for general host and network hardware and topologies remains an open problem. The problem is further exacerbated by perpetual across-the-stack advances in ML deployments: to accommodate larger model sizes, ML deployments are using increasingly large numbers of GPUs [8, 11–13]; to improve hardware utilization, ML deployments are evolving from single-collective jobs to multi-collective jobs to multi-collective multi-tenant jobs sharing host and network resources [4, 24]; and both intra-host and inter-host network interconnects are evolving in terms of scale, hardware functionalities and topologies [11, 20, 25, 26]. With every advance—in scale, in concurrent collectives, in hardware functionality, in intra-host and inter-host topologies—it is imperative to design (presumably) new collective communication algorithms that are optimal for the new host and network hardware and topology.

Put together, the importance of collective communication and perpetual across-the-stack advances in ML deployments lead to a new exciting research direction: synthesizing collective communication algorithms that are optimal with respect to a given host and network hardware and topology. Existing works along this direction give up on optimality [5, 9, 10, 14, 15, 21, 24, 26], scalability [3, 15], and/or generality in terms of the host and network hardware that they can model (Table 1, §8).

This paper presents OptCCL, a technique to synthesize collective communication algorithms that are optimal for a given host hardware and network topology. OptCCL is general: it enables synthesizing optimal algorithms for a general class of collectives (including all existing collectives), for a large class of existing and future hardware (*e.g.*, host hardware with and without specialized intra-host interconnects, heterogeneous network bandwidths within and across hosts, network switches with and without multicast functionality, network switches with and without computation capabilities, etc.), and for multiple concurrent collectives sharing host and network resources.

OptCCL embodies a technical result that establishes a powerful structure in optimal collective communication algorithms: for the objective of minimizing the completion time for collective(s), it is possible to decouple the spatial subproblem of determining the path taken by data between GPUs from the temporal subproblem of scheduling data at each link (*e.g.*, at the granularity of fixed-sized chunks, as in modern collective communication libraries [9, 10]), while maintaining optimality. More specifically, we establish that, when the objective is to minimize the completion time for collective(s), the core optimization problem underlying the goal of synthesizing optimal collective communication algorithms is the spatial subproblem—once we identify the path taken by the data in an optimal solution, scheduling data at each individual link while maintaining optimality is straightforward. This result enables OptCCL (and any other technique) to transform an optimal solution to the spatial subproblem into an optimal end-to-end collective communication algorithm.

OptCCL builds upon the broad class of classical multicommodity flow-based formulations, but uses a novel formulation specialized for collectives. Specifically, rather than treating each sender as a commodity and modifying flow conservation constraints to capture collective structure as would be natural in classical multicommodity flow-based formulations, OptCCL treats each sender-destination pair as a separate commodity. This allows OptCCL to use standard flow conservation constraints; to capture the collective structure,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGCOMM '26, Denver, CO, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2467-1/26/08
https://doi.org/10.1145/3789240.3829207

Table 1: Existing techniques on synthesizing collective communication algorithms. A ⋆ indicates that the corresponding work should be able to support the particular property with natural extensions. Detailed discussion in §8.

	Optimality	Scalability	Generality		
			Set of collectives	Hardware	Multiple collectives
NCCL [9], RCCL [10]	✗	✓	✓	✓	✓
Blink [24]	✗	✓	✗	✗	✓
TACCL [21]	✗	✓	✓	✗	✗
TACOS [26]	✗	✓	✗	✗	✓
SyCCL [5]	✗	✓	✓	✗	✗
TE-CCL [15]	✓	✗	✓	✓⋆	✓⋆
SCCL [3]	✓	✗	✓	✓⋆	✓⋆
ForestColl [28]	✓	✓	✗	✗	✗
OptCCL	✓	✓	✓	✓	✓

OptCCL instead introduces capacity constraints on certain subsets of flows. This formulation admits a nicely structured linear programming (LP) relaxation. Interestingly, we show that for a class of host and network hardware and topologies (that we precisely characterize and that includes many realistic deployments), this LP relaxation already synthesizes optimal solutions. For inputs outside this class, OptCCL iteratively strengthens the relaxation to synthesize optimal collective communication algorithms.

To further improve the time taken to synthesize an optimal solution, OptCCL introduces a technique to exploit the symmetry in the underlying host and network hardware and topology (and the collective). We prove that one of the optimal solutions to the formulation has a particular symmetry, defined with respect to the symmetry in the underlying host and network hardware and topology and the collective; thus, the time taken to synthesize an optimal solution can be improved by reducing the search space to explore only such symmetric solutions.

OptCCL's formulation enables simultaneous optimization for multiple performance objectives. Specifically, for a given input, there may be multiple solutions all of which are optimal with respect to one performance objective (*e.g.*, latency and/or throughput) but may have different performance with respect to other objectives (*e.g.*, amounts of aggregate data transfers). OptCCL enables synthesizing collective communication algorithms that optimize for secondary objectives, while being optimal for the primary objective. This feature enables OptCCL to synthesize optimal algorithms for the case of multiple concurrent collectives within or across tenants (optimality is defined in terms of the primary objective of time taken to complete all collectives).

We evaluate OptCCL across multiple host and network hardware and topologies, with single collectives, and with multiple collectives. Across most of the evaluated scenarios, state-of-the-art techniques are either unable to synthesize algorithms for more than 16–64 GPUs within 3 hours [5, 15] or synthesize algorithms that can be more than an order of magnitude worse than optimal [21]. OptCCL significantly improves upon the current state of affairs: it synthesizes the optimal algorithm for hundreds of GPUs within tens of minutes for all evaluated scenarios.

An implementation of OptCCL is available at https://github.com/ml-collectives/optccl.

2 Preliminaries

Input topologies consist of components (GPUs, CPUs, memory, and switches) and a set of links connecting components. Each (full-duplex) link incident on a GPU, CPU and/or switch has a bandwidth constraint. The links incident on memory components are non-full-duplex [23]; these links have an aggregate bandwidth constraint to accurately capture memory bandwidth.

We define four operations on data. Data can be *stored*. Data can be *transmitted*, that is, the data is forwarded on one of the outgoing links. Data can be *copied*, that is, potentially multiple copies of data at any component may be transmitted over different outgoing links. Data can be *reduced* according to a reduce operation, where two (or more) pieces of data, each of size M, are combined into a single piece of data of size M.

Our formulation enables each component to be capable of doing any subset of the above operations (§5.1). Our formulation also enables an extremely general definition of collectives—informally, any collective that can be described in terms of the four data operations defined above can be incorporated within our formulation (§5.2). Our formulation also enables multiple such collectives concurrently coexisting within the system, within and across tenants (§5.3).

From topologies to graphs. We model the input topology as a directed graph G; see Figure 1 for an example. Each vertex in G corresponds to a component. We let V_b, V_c and V_r be the set of vertices corresponding to components that can store, copy and reduce data, respectively, and V_o and V_d be the origin and destination vertices determined by the collective and the components on which the collective runs. Importantly, our formulation allows any component to be in one or more of the vertex categories (*e.g.*, V_b, V_c, V_r, etc.).

Each edge in G corresponds to a link in the topology; if the link is bidirectional, we create an edge in both directions. To capture the ability of vertices in V_b to store data, for every $v \in V_b$, we also create a self-loop with no bandwidth limit. We capture all bandwidth limits through a single mechanism: for any set of edges L with an aggregate bandwidth limit, we define $b(L)$ to be the bandwidth for those edges, which represents how much data can be sent through the edges in L in one unit of time. We let $\mathcal{L}$ be the collection of all edge sets that have a bandwidth bound. Taking L to be a single edge recovers the bandwidth limit for full-duplex links, and taking

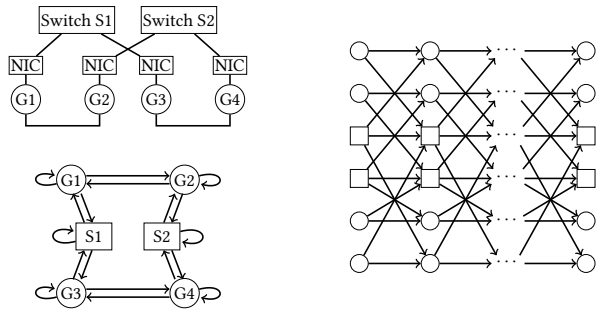


Figure 1: (top left) shows an input topology with 4 GPUs (G1, G2, G3 and G4) across two hosts—each with specialized intra-host interconnect—connected via an inter-host interconnect organized along a rail-based topology. (bottom left) depicts the transformation from the input topology to OptCCL's directed graph representation. (right) shows the corresponding time-expanded network. See §2.

L to be the set of edges incident to a memory component recovers memory bandwidth. See Figure 1 for an example.

Feasible algorithms, performance objectives, and optimal algorithms. OptCCL takes as input the topology (along with the capability of each component), the collective definition, the collective data size and a user-defined chunk size. OptCCL outputs a collective communication algorithm that specifies a set of chunks, a path for each chunk, and for each link along the path, an ordering (or, the transmission times) for chunks transmitted on that link. A collective communication algorithm is said to be feasible if it correctly executes the desired collective operations, while respecting the bandwidth constraints along each link (or set of links).

Our primary objective is end-to-end collective completion time, defined as the time elapsed from the start of the collective (when the first chunk is transmitted) to the time when the collective is completed (the last chunk reaches its destination). A collective communication algorithm is said to be optimal if, for a given input, it achieves the minimum collective completion time over all feasible collective communication algorithms for that input.

There may be multiple optimal algorithms for any given input. OptCCL allows synthesizing, among all such algorithms, an algorithm that also optimizes for a secondary objective. Any secondary objective that is a weighted sum of the amount of data transmitted along all links in the topology can be chosen. To realize this, internal to our formulation, we also use costs c_e for every edge $e \in E$; these costs indicate the cost of transmitting one unit of data along respective edges, and allow us to simultaneously optimize for the secondary objective while guaranteeing optimality with respect to the primary objective. For example, if we want our secondary objective to minimize the total data entering the network, we can set $c_e = 1$ for edges between NICs and switches, and $c_e = 0$ for all other edges. By default, we will set our secondary objective to minimize the total data uploaded to and downloaded from GPUs; thus, we set $c_e = 1$ for edges incident to a GPU, and $c_e = 0$ for all other edges. We will discuss how this secondary objective allows us to efficiently optimize for multiple concurrent collectives.

Time-expanded networks. To model routing of data across components and links over time, we use the representation of so-called time-expanded networks; see Figure 1 for an example. Let G be the input topology, and let T be a constant (we provide an interpretation for T inline, but intuitively it will represent some notion of time). Then, we construct a time-expanded network G^T with T "layers" as follows. We create $(T+1)$ copies of each vertex v in G, labeled as $(v,0), (v,1), \ldots, (v,T)$. For each edge (v_1, v_2) in G, we draw directed edges from vertex (v_1, t) to $(v_2, t+1)$ for $t \in \{0, \ldots, T-1\}$. For vertices $v \in V_b$ that have a self-loop, we also create an edge from (v,t) to $(v,t+1)$. We use V^T and E^T to refer to vertices and edges, respectively, of G^T. We will use the following notation to simplify conversions between G and G^T. For edge $e^T = ((i,t),(j,t+1)) \in E^T$, we use $\tau(e^T) = t$ to denote the time at the tail of e^T and $\mu(e^T) = (i,j) = e$ to denote the corresponding edge in G. Similarly, we use $\kappa(e,t) = \kappa((i,j),t) = ((i,t),(j,t+1)) = e^T$ to convert from e to the corresponding edge in G^T at step t. And analogously for vertices: for $v^T = (v,t) \in V^T$, let $\tau(v^T) = t$ and $\mu(v^T) = v$. We will not describe the bandwidths for edges in G^T; instead, when needed, we will refer back to edges in G.

3 The Optimality-Scalability Tussle

We start with a general formulation for synthesizing optimal collective communication algorithms. The formulation *jointly* solves two problems: routing of data between every sender-destination pair (potentially along multiple paths), and scheduling of data at each individual link at each individual step. The formulation produces optimal solutions, but is not scalable in general. We then consider a natural linear programming relaxation of the formulation that also jointly solves the routing and the scheduling problem; we discuss that this relaxation improves scalability compared to the exact formulation, but struggles with the same tussle: it is possible to achieve either optimality or scalability, but not both. These formulations provide the foundation for our optimal, scalable and practical solution in the next section. For brevity, we focus on the AllGather collective for the case where all components can perform data copies; in §5, we generalize this to incorporate scenarios where some components cannot perform copies, and to collectives that require reductions.

Our formulation builds upon the classical multicommodity flow formulations [2, Ch.17]: we are given a set of distinct commodities; each commodity has an origin and a destination vertex, and a positive demand indicating how much flow must be sent from the origin to the destination vertex. Each commodity/flow must individually satisfy flow conservation constraints; collectively, they must satisfy capacity constraints along each shared edge.

An exact integer programming formulation. For this formulation, we will assume that all transmissions happen in a sequence of T discrete time steps, each of size (or alternatively, of length of time) s. Consider a fixed number of steps T and step size s. We start with the case when all data between a sender-destination pair traverses along a single path. Our formulation creates a commodity between every origin $V_o^T = \{(v,0) : v \in V_o\}$ and every destination $V_d^T = \{(v,T) : v \in V_d\}$ vertex. Specifically, our formulation uses flow variables $f_{e,o,d}$ for every edge $e \in E^T$, and every commodity with origin-destination pair (o,d). For these commodities, flow conservation constraints on the f variables as in classical multicommodity flow formulation ensure that data is routed correctly

through G^T:

$$\sum_{e\in\delta^-(v)} f_{e,o,d} - \sum_{e\in\delta^+(v)} f_{e,o,d} = 0 \quad \text{for } o \in V_o^T, d \in V_d^T, v \in V^T \setminus \{o,d\}; \tag{1}$$

$$\sum_{e\in\delta^-(o)} f_{e,o,d} - \sum_{e\in\delta^+(o)} f_{e,o,d} = 1 \quad \text{for } o \in V_o^T, d \in V_d^T; \tag{2}$$

$$\sum_{e\in\delta^-(d)} f_{e,o,d} - \sum_{e\in\delta^+(d)} f_{e,o,d} = -1 \quad \text{for } o \in V_o^T, d \in V_d^T. \tag{3}$$

To capture collective structure, our formulation introduces additional *usage variables* $w_{e,o}$ for every edge $e \in E^T$ and every origin vertex $V_o^T = \{(v,0) : v \in V_o\}$ in G^T; these usage variables will allow us to capture the collective structure by constraining the bandwidth required to support the corresponding flows in our formulation. [1]

The relationship between f and w is specific to the collective. For example, for the AllGather collective, data sent to all destinations from the same origin is identical; when all components can perform data copies, no component must transmit multiple copies of the same data along a particular edge in an optimal solution. Therefore, it is possible to lower bound the usage w with the constraint:

$$w_{e,o} \geq f_{e,o,d} \qquad \text{for } e \in E^T, o \in V_o^T, d \in V_d^T. \tag{4}$$

Importantly, instead of setting our usage to be the sum over all flows, as would be natural, we allow our usage to be less to account for the fact that the flows may share data. We just ensure that our usage is at least the maximum flow amount, which is achieved by the constraint. Finally, for every set of links L with corresponding bandwidth limitation $b(L)$, the total usage on all links in L by data from all origins must be restricted by $s \cdot b(L)$ for each step of size s:

$$\sum_{e\in L}\sum_{o\in V_o^T} w_{\kappa(e,t),o} \leq sb(L) \quad \text{for } L \in \mathcal{L}, t \in \{0,\ldots,T-1\}. \tag{5}$$

Using the per-edge cost (for secondary objectives, §2) and imposing integrality constraints on f, we get:

$$\min_{f,w} \quad \sum_{e\in E}\sum_{t=0}^{T-1} c_e \sum_{o\in V_o^T} w_{\kappa(e,t),o} \quad \text{s.t.} \quad (1)-(5);\ f \in \{0,1\}, w \geq 0. \tag{6}$$

Importantly, the objective in the formulation does not explicitly use the primary objective (collective completion time) but rather uses the secondary objective; this is because, for a fixed s and T, the collective completion time is predetermined, and the formulation finds the solution with minimum secondary objective value among all solutions with the predetermined collective completion time. As we discuss below, the primary objective is optimized by solving the above formulation over all possible values of s and T.

[1] This is the core difference between OptCCL and prior attempts on synthesizing collective communication algorithms based on multicommodity flow-based formulations, *e.g.*, [15]. Prior attempts modify the classical multicommodity flow formulation to capture the collective structure within the flow constraints. OptCCL uses the standard flow constraints, and captures the collective structure in these usage variables.

It is straightforward to modify the above formulation to consider solutions where data between a single sender-destination pair may traverse across multiple paths. Specifically, let H be the number of uniform-sized chunks originating at a sender. The formulation now creates H commodities between every origin-destination pair, and each commodity is treated independently except for in the bandwidth constraints and in the objective function. The bandwidth constraints are scaled appropriately so that one unit of flow now corresponds to $1/H$ units of data. The formulation, denoted as MILP(17), is provided in §A.1.

Theorem 1. *MILP*(6) *and MILP*(17) *are exact formulations for AllGather for single and for H chunks, respectively.*

The proof of Theorem 1 is in §A.2. As we will discuss in §5, MILP(6) and MILP(17) formulations are very general—they enable each component to be able to do any operation, bandwidth constraints on edges or set of edges, and a general class of (multiple concurrent) collectives. They also directly output the route taken by each chunk and the optimal schedule: the chunk that is transmitted on each link at each time step. However, the computational complexity of generating an optimal solution is high: not only is solving MILP computationally hard, synthesizing an optimal collective communication algorithm requires solving many MILP instances to explore optimal values of s and T. [2]

A linear programming formulation. We now consider a natural step-based relaxation of the integer program MILP(6), one that relaxes the integrality constraints on f:

$$\min_{f,w} \quad \sum_{e\in E}\sum_{t=0}^{T-1} c_e \sum_{o\in V_o^T} w_{\kappa(e,t),o} \quad \text{s.t.} \quad (1)-(5);\ f, w \geq 0. \tag{7}$$

We use the notation LP(7)$[s,T]$ to indicate formulation (7) with step size s and number of steps T when s and T are not obvious. This formulation maintains the generality of MILP(6) and MILP(17); and, while the process of creating a chunk-based schedule is less obvious than for MILP(6) and MILP(17), it is possible (we discuss in §4).

The core challenge is the tussle between optimality and scalability. Specifically, allowing fractional flow values means that we no longer require integer flow constraints and thus LP(7) can be solved in polynomial time [2, Ch.17] for any given step size s and number of steps T; however, the absolute time taken to solve LP(7) grows quickly with T—this is because both the size of the time-expanded network G^T and the number of variables in the LP grow roughly linearly with T. Thus, we want a small T. Ideally, we also want a small s: this enables efficient pipelining of data along the path, keeping every link continuously busy; indeed, suboptimality of imperfectly pipelined solutions grows linearly with s (or, more precisely, inversely proportional to T since optimal end-to-end

[2] An optimal solution may require sending more than one chunk per time step s; thus, identifying an optimal value of s requires exploring potentially $|E|$ many values (one for each unique link bandwidth). An obvious upper bound on T is the product of total number of chunks (the ratio of collective data size to chunk size) and number of components, since any given chunk must never traverse a vertex in the graph multiple times in an optimal solution; we can search over all values of T by doubling until we find a feasible solution, then doing binary search to determine the smallest feasible value. Synthesizing the optimal solution thus requires solving the MILP as many as $|E| \times \log(|V| \times (\text{collective data size}/\text{chunk size}))$ times.

collective completion time is $s \times T$). Unfortunately, since s and T are tightly coupled in these formulations, we can achieve either scalability (a small T) or optimality (a small s), but not both.

Take-away. The two formulations above—the exact integer programming formulation and the linear programming relaxation—suffer from a tussle similar to existing works [3, 5, 15, 21]: it is possible to achieve either optimality or scalability, but not both. We believe this tussle is fundamental to an inherent property of all of these formulations: by incorporating individual steps within the formulation (*e.g.*, via variable s), they tightly couple the spatial problem of routing data between sender-destination pairs (*e.g.*, via constraints (1)–(4) that capture the path taken by data) with the temporal problem of scheduling data on individual links at each time step (via constraint (5) that captures the scheduling of data while maintaining bandwidth constraints in each step). In the next section, we present a formulation that is able to break this tussle, while maintaining the generality of the above formulations.

4 OptCCL

This section presents a formulation that maintains the generality of MILP(6) and MILP(17), and yet, enables scalable syntheses of optimal collective communication algorithms. The key technical result underlying this formulation is a structure in optimal collective communication algorithms: for our primary objective of collective completion time, it is possible to decouple the spatial subproblem of determining the path taken by data between sender-destination pairs from the temporal subproblem of scheduling data at each link at each time step, while maintaining optimality. In fact, we will establish a more powerful result: for our primary objective of collective completion time, the core optimization problem underlying the goal of synthesizing optimal collective communication algorithms is the routing problem—once we identify the path taken by the data between all sender-destination pairs in an optimal solution, scheduling data at each individual link in each individual time step while maintaining optimality is straightforward.

OptCCL will decouple the spatial and the temporal subproblems—it will generate the path taken by data in an optimal solution using a formulation that combines constraints (1)–(4) along with mild bandwidth constraints sufficient to later solve the temporal problem, while maintaining optimality. Specifically, rather than enforcing bandwidth constraints for each individual step as in (5), our formulation will now impose constraints on *average bandwidth* used by the solution for any link over the entire lifetime of the collective. We will then show that it is possible to take an optimal routing solution—the path taken by data between every sender-destination pair in a manner that bandwidth constraints are satisfied *on an average* rather than on a per-step basis—and construct a step-based schedule over uniform-sized chunks (that is, scheduling data at each individual link at each individual time step at the granularity of uniform-sized chunks) while maintaining feasibility and optimality.

4.1 Decoupling data routing and scheduling

Let T be an upper bound on the total number of transmissions, across all sender-destination pairs, any piece of data might require to reach from its sender to its destination (note that data may not necessarily traverse along the shortest path). Let R be a desired bound on the completion time of our solution. Then we modify constraint (5) in LP(7) to require that bandwidth constraints are imposed on average bandwidth during the lifetime R of the collective:

$$\sum_{t=0}^{T-1} \sum_{e \in L} \sum_{o \in V_o^T} w_{\kappa(e,t),o} \leq Rb(L) \qquad \text{for } L \in \mathcal{L}. \tag{8}$$

Then our new formulation becomes:

$$\min_{f,w} \quad \sum_{e \in E} \sum_{t=0}^{T-1} c_e \sum_{o \in V_o^T} w_{\kappa(e,t),o}$$
$$\text{s.t.} \quad (1)-(4), (8);\ f, w \geq 0. \tag{9}$$

We use the notation LP(9)$[R, T]$ to indicate formulation (9) with desired completion time R and number of steps T.

The above formulation, unlike LP(7), solves only the spatial subproblem: it determines the amount of data transmitted on each path between any sender-destination pair in a manner that bandwidth constraints on each link are satisfied on an average, but does not specify the schedule for each chunk of data on each link at each time step. Nevertheless, we prove the following theorem:

Theorem 2. *LP(9) is equivalent to LP(7) in the following sense: given a solution to LP(9)$[R_1, T_1]$, there are values s_1 and T_1' such that we can construct a solution to LP(7)$[s_1, T_1']$ of equal cost and with completion time $s_1 T_1' < R_1 + \epsilon$ for any $\epsilon > 0$. And given a solution to LP(7)$[s_2, T_2]$ with completion time $s_2 T_2$, we can construct a solution to LP(9)$[s_2 T_2, T_2]$ of equal cost.*

The full proof of Theorem 2 is in §A.3. This theorem establishes that we can convert between solutions for LP(7) and LP(9) with identical objective cost and identical completion time (up to an arbitrarily small ϵ term). Thus, we can find an optimal solution for LP(7)—over all possible choices of s—by solving LP(9). This is powerful because it demonstrates that the optimal collective communication algorithm (in fact, the entire solution space) does not change even if one ignores the problem of scheduling individual data chunks; solving the spatial subproblem is the crux.

Choosing R and T. We first determine R, the optimal completion time for our collective algorithm, which we can do independently from T. A simple upper bound for R is the size of all the output data divided by the smallest bandwidth in the input; since choosing R too small makes the LP infeasible, it is possible to compute it via bisection search. We instead use a better strategy from [28]: R can be computed directly by solving several maximum flow problems on a modified version of G (details in §A.4). Once R is determined, we determine T that upper bounds the length of the longest path that any piece of data travels in an optimal solution. A trivial upper bound on this value is the number of edges in G. This means that via repeated doubling and bisection search, we can determine the best value of T in roughly $\log |E|$ attempts.

4.2 Synthesizing an optimal routing solution

LP(9) (as well as LP(7)) are relaxations of the exact formulation MILP(6) and MILP(17); thus, a feasible solution to the LP does not necessarily imply an optimal routing solution. To understand this, let us call a solution *LP-feasible* if it satisfies the constraints of the LP at hand. The solution is also *algorithm-feasible* if the schedule

it prescribes (using the technique discussed in §4.3) satisfies two conditions: every piece of data reaches all of its destinations, and the data crossing any link in any step never exceeds the bandwidth of the link. Below, we characterize host and network hardware and topologies for which an LP-feasible solution to LP(9) is also algorithm-feasible (that is, provides an optimal routing solution); and when such is not the case, a mechanism to synthesize an algorithm-feasible solution from a given LP-feasible solution.

Algorithm-feasibility is exactly what MILP(6) and MILP(17) encode: thus, the notions of LP-feasibility and algorithm-feasibility coincide for MILP(6) and MILP(17). However, they can diverge for our relaxations. This is because usage variables in constraint (4) are allowed to be less than the sum of the corresponding flow values based on an optimistic assumption about the structure of the optimal solution: all flows sharing that link carry identical data and can be merged into one transmission. When the optimal solution does not have that structure, the usage variables may underestimate the bandwidth required (by the step-based schedule corresponding to the LP-feasible solution) on one or more edges.

LP(9) already synthesizes optimal routing solutions for a large class of host and network hardware and topologies. We now establish that, for a class of input topologies that we call ideal topologies, the LP(9) relaxation does produce an algorithm-feasible solution. We define ideal topologies to be the class of topologies in which (1) in an optimal solution, all data traverse no more than four edges in G (that is, four physical links in the network); and (2) the bandwidth limits and objective costs are imposed on inter-host links (that is, links between switches and NICs). For ideal topologies, we prove the following theorem:

Theorem 3. *For ideal topologies, LP(9)-feasible solutions are also algorithm-feasible. Thus, for ideal topologies, LP(9) synthesizes an optimal routing solution.*

The full proof is in §A.5. We provide some intuition on ideal topologies. Many existing deployments that use specialized intra-host interconnects (*e.g.*, NVLinks) have a NIC connected to each GPU with bandwidth much lower than the bandwidth of NVLinks connected to the GPU; in such cases, the bandwidth limits and objective costs are imposed on inter-host links. Now, suppose such is the case. Then, Theorem 3 implies that any topology where all paths between each pair of GPUs have no more than four physical links is an ideal topology. For example, a deployment where all GPUs within a host are connected via NVLinks and where GPUs across hosts are connected along a rail-based network topology, with a single layer of rail switches, satisfies this condition when all data transfers use the shortest path since such paths have at most four links: GPU→NIC→switch→NIC→GPU.

Synthesizing optimal routing solutions for non-ideal topologies. For non-ideal topologies, an LP(9)-feasible solution *may* not be algorithm-feasible. We can nonetheless synthesize an optimal algorithm by repeatedly strengthening the relaxation, using the value of algorithm-infeasible optimal solution as a lower bound.

We first solve LP(9), and check if the resulting solution corresponds to an algorithm-feasible one of the same cost. To perform this check, we convert the routing solution into a concrete set of trees (we describe this construction in §4.3). Each tree describes how data from one origin travels to all its destinations. Whenever a vertex in the tree has multiple outgoing edges, the data is considered to be copied. We then check for each edge whether the sum of flow values (recall, an edge may be in multiple trees) is no more than the corresponding usage value. If the check succeeds, the solution is algorithm-feasible and we have found an optimal routing solution—indeed, the optimal value to any relaxation is a lower bound on the optimal algorithm objective value.

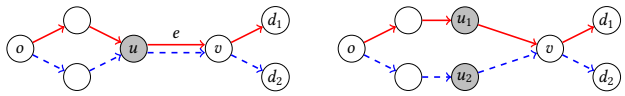


Figure 2: An example to illustrate scenarios where LP-feasible solutions may not be algorithm-feasible (left) and the vertex duplication mechanism used in OptCCL to resolve such cases (right).

When the check fails, there is some edge $e = (u, v)$ in G^T whose trees after converting the routing solution carry distinct pieces of data from a common origin o. Figure 2 (left) shows such a case in which two flows send one unit on edge e, but the LP assigns only one unit of usage value under the optimistic assumption that these data can be transmitted in a single unit (and be copied later). However, the two flows reached e along different paths and carry distinct data, so the optimistic assumption made by the LP does not hold: e must actually send two units.

To prevent this phenomenon from occurring, we strengthen the formulation: we duplicate the offending vertex u at the tail of e, splitting it into copies indexed by the destinations its outgoing data is bound for, so that the current solution will no longer be LP-feasible (*e.g.*, Figure 2 (right) splits u into u_1, u_2, forcing the two flows onto separate edges). [3] After duplicating vertices for every edge whose usage was less than the sum of the corresponding flow values, we end up with a slightly larger time-expanded network and re-solve LP(9) on it. We repeat until the solution is algorithm-feasible, and since all our relaxations are lower bounds, it is also provably optimal. We emphasize that this iterative strengthening procedure is only performed if necessary.

4.3 Synthesizing an optimal schedule

The optimal routing solution from the previous subsection specifies, for each sender-destination pair, the set of paths taken by data and the amount of data transmitted on each path. In this subsection, we describe an efficient mechanism to convert the optimal routing solution into an optimal schedule that specifies, for each sender-destination pair, data transmitted on each link at each time step.

The mechanism takes as input the host and network hardware and topology, the routing solution from the previous subsection, as well as user-defined chunk size and total data size. To maintain practicality (*e.g.*, existing collective communication libraries require algorithms that transmit data in discrete, *uniform-sized* chunks)[4], our mechanism will ensure that all transmissions are performed at

[3] This is the same technique as the exact, exponentially sized formulation of §A.6, which preemptively duplicates each vertex an exponential number of times; here we apply it lazily, duplicating only the few vertices needed so that the current solution is no longer feasible in the enlarged graph.

[4] Given a routing solution, one possibility to generate an optimal schedule is to use the construction from the proof of Theorem 2; however, this construction does not guarantee that all chunks will be of uniform sizes (in other words, maintaining the optimality of the routing solution may require chunks of non-uniform sizes).

the granularity of uniform-sized chunks. This schedule is the final collective communication algorithm.

At a high level, the mechanism works as follows. It starts with the routing solution, and performs a so-called path decomposition: for each sender-destination pair, it generates a set of paths used for data transmission in the routing solution (that is, paths along which the flow value for the corresponding sender-destination pair is non-zero). The mechanism then considers each sender independently, and aggregates the paths from this sender to all destinations into a collection of trees, each rooted at the sender; importantly, a path (and an edge) may appear in one or more trees. For each tree τ, the mechanism assigns a volume x_τ representing the amount of data that must be transmitted along that tree; indeed x_τ must satisfy several constraints: (1) the sum of x_τ should match the total data size to be transmitted from the sender (that is, all data from the sender must be transmitted); and (2) for each edge, the sum of x_τ across all trees that include that edge must not exceed the fraction of data assigned to that edge (that is, total data size × flow value for that edge) in the routing solution. We show that, given a routing solution, it is trivially possible to generate such set of trees; our mechanism aims to construct a set of trees with a small cardinality.

Given this set of trees, we use an adaptation of weighted fair queueing [18, 19] to construct a schedule over uniform-sized chunks. The goal is to faithfully emulate the routing solution in which each tree τ continuously carries x_τ units of data using only discrete transmissions of chunks of a fixed size C. To do so, we pick an integer parameter K that controls how finely we discretize time, and require each tree to release its chunks at a rate of $x_\tau/(KC)$ per step, so it transmits all of its data over K steps. These chunks then travel down their tree, and at each edge they contend with chunks from other trees that share that edge. To resolve this contention, we use the idea of so-called virtual time from [18, 19]: virtual time of a chunk represents the time when all data corresponding to the chunk would finish transmitting on the edge in the LP(9) solution. Each edge now maintains a queue of waiting chunks; at every step, it forwards chunks with the smallest virtual time. Intuitively, by always serving the chunks that have fallen furthest behind the LP(9) optimal routing solution, no chunk is allowed to lag too far behind. Hence, the resulting uniform-chunk schedule finishes only slightly later than the optimal LP(9) routing solution.

We provide full details in §A.7. Let $x_{\min} = \min_\tau x_\tau$ be the smallest amount of data assigned to any tree in our decomposition; recall that T is an upper bound on the total number of transmissions, across all sender-destination pairs, any piece of data might require to reach from its sender to its destination, that $b_{\min}$ is the minimum bandwidth in the input topology, and that C is the fixed chunk size. Then, we establish that it is possible to find a $K = K^\star$ such that the following theorem holds:

Theorem 4. *Given a routing solution with collective completion time R, OptCCL synthesizes a collective communication algorithm with uniform-sized chunks with collective completion time at most*

$$R\,(1+\gamma)\left(1+\gamma+\frac{CT}{x_{\min}}\right)$$

where, $\gamma = \sqrt{2CT/Rb_{\min}}$.

We prove a better, albeit more non-intuitive, bound in §A.7. The theorem suggests that, for inputs where the slowest link is capable of transmitting data much larger than $C \times T$ during the lifetime of the collective ($R \times b_{\min} \gg C \times T$) and for routing solutions that admit each tree in the decomposition transmitting data much larger than $C \times T$ ($x_{\min} \gg C \times T$), it is possible to convert the optimal routing solution into a collective communication algorithm with uniform-sized chunks in a manner that the optimal collective completion time is inflated by a tiny amount. Note that both C and T are expected to be tiny; thus, the two conditions are likely to hold for most realistic deployments and inputs. Our evaluation in §7 suggests that this is indeed the case: OptCCL synthesizes a schedule that is within 1% of the optimal collective completion time for all evaluated scenarios. Moreover, technology trends are favorable: we find that the inflation in latency reduces with collective sizes (since R and $x_{\min}$ increase roughly proportionally) and with scale (since R increases more rapidly than T).

5 OptCCL generality

Our discussion so far has focused on an input topology with all components capable of performing copy operations and on a single AllGather collective. In this section, we provide details on OptCCL generality: using OptCCL to synthesize optimal algorithms for different hardware functionalities (§5.1), for different collectives (§5.2), and for multiple concurrent collectives (§5.3).

5.1 OptCCL for different hardware functionalities

OptCCL enables specifying the capability for each individual component in terms of the subset of four operations (§2)—transmit, store, copy, reduce. Our LP(9) formulation assumes that all components are capable of transmit, store and copy operations. In particular, it (correctly) enforces that multiple copies of the same data are never sent along a single link; in other words, it enforces the constraint that the bandwidth usage of a link is bounded by the maximum of the f variables. This constraint may not necessarily hold when one or more components are not capable of performing copies.

We now extend our formulation to scenarios where some components may not support the copy operation (*e.g.*, switches that do not support multicast). We start by explaining an approach similar to the one used for synthesizing optimal routing solutions for non-ideal topologies (§4.2)—using duplicate vertices—to handle scenarios when some components are not capable of performing copies; this approach may unnecessarily impact scalability even when the underlying topology is an ideal topology. We then present an approach that achieves the same goal as the general approach, but duplicates vertices only when needed—that is, when an LP-feasible solution is not algorithm-feasible. Components that do not support reduce operations are handled symmetrically by reversing solutions, as we discuss in §5.2.

Formulation via duplicate vertices. Recall that the difficulty in handling components not capable of performing copies is that LP(9) is free to route two flows from the same origin carrying different data out of a non-copy vertex v as though v had simply copied one incoming transmission. For example, in Figure 3 (left), two



Figure 3: An example where LP-feasible solutions do not correctly handle components that do not support copy (left) and the strengthening via vertex duplication to resolve such cases (right).

flows carrying different data leave o, share edge e, and split at the non-copy vertex v; the LP sets usage variables for e assuming that it transmits only one unit of data, but because v cannot copy, the algorithm must send both pieces across e separately. We prevent this by making duplicate copies of v. Let $\overline{V_c} = V \setminus V_c$ be the set of vertices in G that do not allow copies. When we build the time-expanded network, we replace each $v \in \overline{V_c}$ with $|V_c|$ copies, one tied to each copy-capable vertex; all of v's incoming edges enter every copy, but each copy's outgoing edges lead only to its associated vertex in V_c. Flows leaving v toward different destinations are thus forced through different copies, and the LP can no longer merge them for free. Using LP(9) on this new time-expanded network G_*^T leads to a routing solution for AllGather on topologies with components that cannot perform copies. For a full description, see §A.8.

Lazy vertex duplication. This approach is practical only if there are few components that cannot perform copies. But in the worst case, this modification will result in almost squaring the number of vertices and edges in our time-expanded network, which makes the size of the formulation grow very quickly. To address this, we will not duplicate vertices up front. We start with LP(9) and add a flow-like constraint on usage variables (w) to LP(9) in an effort to prevent copies at vertices in $\overline{V_c}$:

$$\sum_{e \in \delta^-(v)} w_{e,o} - \sum_{e \in \delta^+(v)} w_{e,o} = 0 \qquad \text{for } o \in V_o^T, v \in \overline{V_c}^T. \tag{10}$$

Unfortunately, there are corner-case examples where adding this constraint by itself does not guarantee that an LP-feasible solution is algorithm-feasible. For such corner-case scenarios, we again use our formulation strengthening technique described in §4.2: solve the formulation, check whether its solution is algorithm-feasible, and if not, strengthen it and re-solve. To strengthen, we duplicate the offending vertex v (in this case, the head of the edge; Figure 3) as described above so that the diverging flows are split across different copies, then re-solve on the enlarged time-expanded network. In the worst case, we might need to create many duplicate vertices; however, lazy duplication improves scalability whenever possible.

5.2 OptCCL for all collectives

OptCCL techniques are general in terms of collectives: OptCCL enables synthesizing optimal collective communication algorithms for any collective that can be described in terms of the four data operations (§2). In this subsection, we describe how to use OptCCL to synthesize optimal algorithms for known collectives: Broadcast, Reduce, ReduceScatter, All-to-All, and AllReduce.

All-to-All. Optimal algorithms for the All-to-All collective require neither copies nor reductions. Thus, OptCCL formulation for this collective is much simpler since usage variables are not required—the amount of data transmitted along an edge can be directly inferred from flow variables. Specifically, an optimal algorithm for the All-to-All collective can be synthesized by changing constraint (8) in LP(9) to

$$\sum_{t=0}^{T-1} \sum_{e \in L} \sum_{o \in V_o^T} f_{\kappa(e,t),o,d} \le Rb(L) \qquad \text{for } L \in \mathcal{L}. \tag{11}$$

Thus, the new linear program is:

$$\min_{f} \quad \sum_{e \in E} \sum_{t=0}^{T-1} c_e \sum_{o \in V_o^T} \sum_{d \in V_d^T} f_{\kappa(e,t),o,d}$$
$$\text{s.t.} \quad (1)-(3), (11);\ f \ge 0. \tag{12}$$

ReduceScatter. ReduceScatter requires reduction operations, and does not require copy operations. Thus, the standard technique for generating solutions for ReduceScatter is to reverse the solutions for AllGather. One assumption that enables this technique is that the set of vertices in G that allow data to be copied are exactly the same set of vertices that allow data to be reduced. Unfortunately, this assumption may not always hold true—while GPUs typically allow both actions, switches may not support copy and reductions, and memory may not support reductions. Fortunately, OptCCL techniques do not require such an assumption: in our formulations, we decouple the ability to store, copy and/or reduce data (via well-defined sets of vertices V_b, V_c and V_r). In particular, we use exactly the same formulation as for AllGather with vertices in V_c replaced with a new set of vertices V_r that are allowed to perform reductions (and reverse direction of all edges in G).

Broadcast and Reduce. Broadcast is the same as AllGather; the only change is in defining the set of origin vertices, with V_o set to be the single vertex that broadcasts the data. Similarly, Reduce is the same as ReduceScatter, where we let V_d be the single vertex that receives the data.

AllReduce. An optimal algorithm for AllReduce may require both copies and reductions of data. Unfortunately, we cannot create an exact formulation for AllReduce by duplicating some of the vertices in our time-expanded network as we could for AllGather or ReduceScatter. This is because the graph created by duplicating switch vertices, say for AllGather, is not symmetric with respect to time—*e.g.*, the in-degree of a duplicate switch vertex is not the same as the out-degree. So duplicating vertices can only support copy operations or reduction operations, but not both simultaneously. We thus introduce a new way to determine the usage on an edge (w) from corresponding flow values (f).

We first establish a structure in optimal AllReduce algorithms: for the objective of collective completion time, there exists an optimal solution where all reductions take place before copies.

Theorem 5. *For the primary objective of collective completion time, the following holds: given an optimal AllReduce algorithm in which any piece of data is copied and later reduced, we can find an optimal algorithm with identical completion time where all pieces of data are first reduced and copied later.*

We emphasize that the theorem does not guarantee that there is an optimal solution with reductions before copies for any arbitrary

objective; it is only for the collective completion time objective. The proof of the theorem, presented in §A.9, establishes a stronger result: we can find an algorithm where the reduction happens before copies with identical collective completion time *and* smaller amount of total data transmitted (where we sum the total amount of data sent over all edges).

Theorem 5 suggests that it may be possible to synthesize optimal AllReduce algorithms within our formulation using the standard approach of implementing AllReduce [9]—performing ReduceScatter (that may require reductions but no copies) followed by AllGather (that may require copies but no reductions). It turns out, though, that naïvely using this standard approach may not be optimal due to two reasons. First, this assumes the intermediate state of the solution, namely that equal amounts of the reduction result are present on each GPU. In non-symmetric topologies, this need not be the best intermediate state. Secondly, although we argued that for each piece of data, the reduce-before-copy criterion should be true, waiting for all reductions to finish before beginning any copy operations may lead to suboptimality. Indeed, it may be possible to benefit by parallelizing the beginning of the AllGather algorithm and the end of the ReduceScatter algorithm.

Now we describe how we modify our existing formulations to address both of these concerns. At a high level, we include a set of flow and usage variables, along with their corresponding flow constraints for each of AllGather and ReduceScatter. To address the first concern, we introduce a variable (z) that describes what portion of the reduction result is completed at each vertex, modifying the demands for the flow constraints accordingly. As in LP(9), we will have a constraint bounding the average bandwidth on every edge, considering usage from both constituent collectives. Finally, when generating a chunk-based schedule, we concatenate ReduceScatter and AllGather on a chunk level: as soon as a chunk completes its schedule for ReduceScatter, it is re-enqueued at the beginning of the AllGather scheduling process. In this way, the resulting algorithm performs reductions before copies for each piece of data while parallelizing ReduceScatter and AllGather as much as possible.

See §A.9 for the full formulation and proof of its optimality.

5.3 OptCCL for multiple concurrent collectives

OptCCL is able to synthesize optimal algorithms for any arbitrary combination of collectives, each operating on any subset of vertices, as long as the primary objective is to optimize *makespan*: the total time required to complete all collectives.

To handle concurrent collectives, we use the same approach as in the AllReduce collective (§5.2) with the difference being that we now allow the collectives to run concurrently rather than concatenating them. In particular, we create a unified formulation as follows. For each constituent collective, we introduce a set of flow variables, usage variables (if necessary), and corresponding flow constraints. Then, aggregating over variables from all collectives, we write shared objective and bandwidth constraints (similar to (8)) that bound the average bandwidth usage on each edge. From the resulting optimal routing solution, we produce an optimal chunk-based schedule as if there was a single collective (§4.3); thus, all collectives run concurrently, with their combined transmissions on each edge respecting its bandwidth.

6 Exploiting symmetry

The sizes of all our linear programs so far are polynomial in the size of our underlying graph G; however, they still may become intractable as the size of the input grows. For example, for AllGather, the number of variables needed in LP(9) grows as $\Omega(n^3)$, where n is the number of GPUs (where flows originate). In this section, we present techniques to improve computation time for synthesizing our optimal routing solution.

Dantzig-Wolfe decomposition [2] is a standard technique used to speed up the process of solving linear programs with certain block structure (that we discuss below). It turns out that our linear programs have that structure: they can be described as many nearly-independent subproblems, each of which restricts the routing of data from a single origin. Applying Dantzig-Wolfe decomposition to our linear programs, we can iteratively converge to an optimal solution by solving each (much smaller) subproblem, albeit with an iteration-specific objective function. Unfortunately, while Dantzig-Wolfe decomposition improves the synthesis time for optimal routing solutions, it remains slow for large inputs.

Below, we present a new technique that we call Mirrored Dantzig-Wolfe that significantly reduces the time taken to synthesize routing solutions. We start by defining a notion of symmetry in the context of our input. We then establish that, if the input is symmetric, we can always find optimal solutions that are also symmetric; and, by constraining our search space to symmetric solutions, each subproblem in our previous Dantzig-Wolfe decomposition is isomorphic—that is, a solution to one subproblem can be mapped to an equivalently feasible solution to each other subproblem. Using this result, we are able to make equivalent progress solving only one subproblem (as opposed to n subproblems) each iteration. The size of each subproblem is roughly a factor of n^2 smaller than the original problem, so we can solve the original linear program and synthesize optimal routing solutions much faster.

Dantzig-Wolfe decomposition. To give a slightly more detailed overview, Dantzig-Wolfe decomposition is a reformulation of structured linear programs as a master problem and subproblems, where variables in each subproblem are only connected by a few linking constraints. For each subproblem, we use the fact that any polytope can be described as a convex combination of its extreme points. In the master problem, we substitute the variables with their description as a convex combination and use the convex combination coefficients as our new variables. This converts our problem into one with only a few constraints, but (exponentially) many variables. We can solve this converted problem using column generation, where we set the objective for each subproblem to try to find new extreme points that improve the solution.

Our formulations are ideal candidates for Dantzig-Wolfe decomposition. For example, consider the AllGather formulation in LP(9). We observe that the variables for each choice of origin $o \in V_o^T$ are mostly independent; that is, nearly all of the constraints only connect variables with the same choice of o. Only the bandwidth constraints involve variables with different o values, and even then, only the w variables are linked, not the more numerous f variables.

Applying Dantzig-Wolfe decomposition to our AllGather formulation, we have a subproblem (S_o) for each $o \in V_o^T$, and we wish to minimize $\sum_{e \in E} \sum_{t=0}^{T-1} \hat{c}_e w_{\kappa(e,t),o}$, subject to:

$$\sum_{e\in\delta^-(v)} f_{e,o,d} - \sum_{e\in\delta^+(v)} f_{e,o,d} = 0 \quad \text{for } d \in V_d^T, v \in V^T \setminus \{o,d\};$$
$$\sum_{e\in\delta^-(o)} f_{e,o,d} - \sum_{e\in\delta^+(o)} f_{e,o,d} = 1 \quad \text{for } d \in V_d^T;$$
$$\sum_{e\in\delta^-(d)} f_{e,o,d} - \sum_{e\in\delta^+(d)} f_{e,o,d} = -1 \quad \text{for } d \in V_d^T;$$
$$w_{e,o} \geq f_{e,o,d} \quad \text{for } e \in E^T, d \in V_d^T;$$
$$f, w \geq 0, \tag{13}$$

where the $\hat{c}_e$ are costs dependent on the master problem solution in the current iteration. To state the master problem, let J_o index the set of extreme points of the polyhedron defined by (S_o). And let $\bar{w}^j_{e,o}$ be the $w_{e,o}$ values at the jth extreme point of the polyhedron. Then our full master problem is:

$$\min_{\lambda} \quad \sum_{e\in E}\sum_{t=0}^{T-1} c_e \sum_{o\in V_o^T}\sum_{j\in J_o} \lambda_o^j \bar{w}^j_{\kappa(e,t),o}$$
$$\text{s.t.} \quad \sum_{t=0}^{T-1}\sum_{e\in L}\sum_{o\in V_o^T}\sum_{j\in J_o} \lambda_o^j \bar{w}^j_{\kappa(e,t),o} \leq Rb(L) \text{ for } L \in \mathcal{L};$$
$$\sum_{j\in J_o} \lambda_o^j = 1 \quad \text{for } o \in V_o^T;$$
$$\lambda_o^j \geq 0 \quad \text{for } o \in V_o^T, j \in J_o. \tag{14}$$

For more details on how to use this decomposition to generate the final solution, please see [2]. Indeed, this technique easily extends to all our formulations.

Mirrored Dantzig-Wolfe. [5] Even with Dantzig-Wolfe decomposition with our formulations, it may take a prohibitively long time to synthesize optimal routing solutions. However, we observe that in many cases our Dantzig-Wolfe decomposition is made up of subproblems that seem remarkably similar, especially when our inputs are symmetric in a sense that we describe below. We next outline how we can leverage this symmetry to greatly improve the time taken to synthesize the optimal routing solutions.

We define symmetry through the concept of indistinguishability. Two GPUs are *indistinguishable* if, after removing all component and link identifiers from our input, the corresponding GPUs are isomorphic; a precise definition can be found in §A.10, but intuitively, isomorphism here is defined with respect to both the input topology and the collective structure. We further introduce a *Symmetry Partition*, $\mathcal{P}$; the sets in $\mathcal{P}$ are disjoint subsets of vertices in V_o (each referred to as an "indistinguishability class") such that for each $P \in \mathcal{P}$ and for any $g_1, g_2 \in P$, g_1 and g_2 are indistinguishable. For each indistinguishability class $P \in \mathcal{P}$, we define a mapping function π_P that maps each component (and each link) in the input topology to one or more components (and edges) in the input topology. Given each of the π_P, we say that a solution to our formulation is symmetric if usage and flow variables on any edge e are the same as the usage and flow variables on all edges to which any of the π_P maps e. Given these definitions, we prove the following theorem:

Theorem 6. *Given a solution to LP*(9)*, Symmetry Partition $\mathcal{P}$, and an associated function π_P for each $P \in \mathcal{P}$, we can find a symmetric solution to LP*(9) *of the same completion time and objective value.*

The proof is presented in §A.10. The existence of a symmetric optimal solution allows us to reduce the size of our formulation by eliminating (now redundant) variables and constraints, while maintaining optimality. Specifically, when integrated with Dantzig-Wolfe decomposition, our symmetry definition along with Theorem 6 allows us to guarantee that for $o_1, o_2 \in V_o^T$, if $\mu(o_1)$ and $\mu(o_2)$ are in the same indistinguishability class, then subproblems S_{o_1} and S_{o_2} are isomorphic; that is, they are identical up to the relabeling of variables and remain identical after each iteration. Therefore, instead of solving $|V_o|$ subproblems each iteration, we only need to solve one subproblem per indistinguishability class ($|\mathcal{P}|$ subproblems). And in the special case where all $v \in V_o$ are indistinguishable, we need only solve one subproblem per iteration. All these scalability benefits can be achieved while maintaining optimality.

7 Evaluation

We now evaluate OptCCL and compare it with state-of-the-art systems—TE-CCL, SyCCL and TACCL—over two topologies: A100 with 8 GPUs per host [7] and DOE with 4 GPUs per host [17]; in both cases, we use a multi-rail inter-host network topology [25].

Setup. We evaluate performance in terms of algorithmic bandwidth, defined as the ratio of collective data size to collective completion time (higher is better). We use a server with 32-core Intel Xeon Gold 6234 CPU and 314GB memory to measure the synthesis time.

We evaluate TE-CCL, SyCCL, and TACCL using their publicly available code. We were unable to run SyCCL on scale-up (single host) topologies, as their implementation appears to be designed exclusively for multi-host settings with a limited set of link types. TACCL has bugs in its All-to-All implementation (as is noted in their GitHub issues). Each of these systems requires, as input, a fixed number of chunks; typically, the synthesis time worsens and the performance improves with number of chunks. We present results for the largest number of chunks for which each system was able to synthesize an algorithm within a three-hour limit.

We set up OptCCL with existing hardware features. That is, GPUs and CPUs have the ability to perform each of the above four operations. Memory can store, transmit and copy, but cannot perform reductions. Switches can store and transmit but cannot perform copies and reductions. For measuring synthesis time, we generate one representative schedule per symmetry class which contains all the same information as a full schedule.

7.1 OptCCL for single collectives

Figure 4 and Figure 5 show results for the two topologies, both with NVLinks, for AllGather and All-to-All collectives, respectively. Across both topologies and collectives, TE-CCL was unable to synthesize algorithms within three hours for more than 16–32 GPUs;

[5] Our Mirrored Dantzig-Wolfe technique may be of general interest to the optimization community working on multicommodity flow problems. In particular, existing decomposition-based techniques in such problems reduce the search space by creating one decomposition for each commodity, and creating a subproblem for each decomposition. We show that such techniques can be improved by creating a decomposition for each symmetry class instead; since each symmetry class contains multiple commodities, this drastically reduces the number of subproblems.

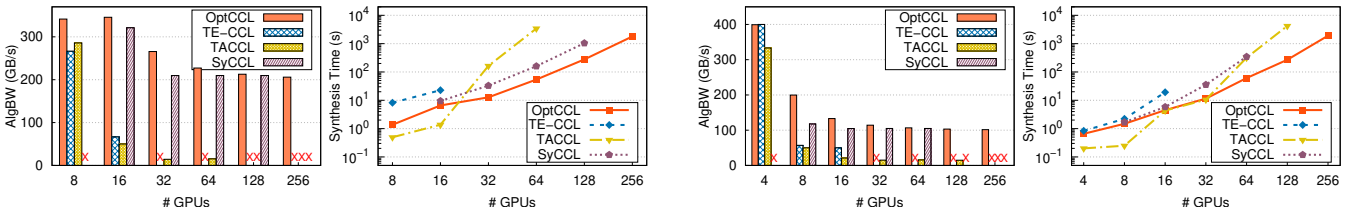


Figure 4: Performance in terms of algorithmic bandwidth and synthesis time for the A100 (left) and the DOE topologies (right) for the AllGather collective. OptCCL synthesizes algorithms that have 1–5×, 1–1.6×, 1.2–18× better performance than TE-CCL, SyCCL and TACCL, respectively. OptCCL also requires 3–60× lower synthesis time than SyCCL and TACCL on large topologies.

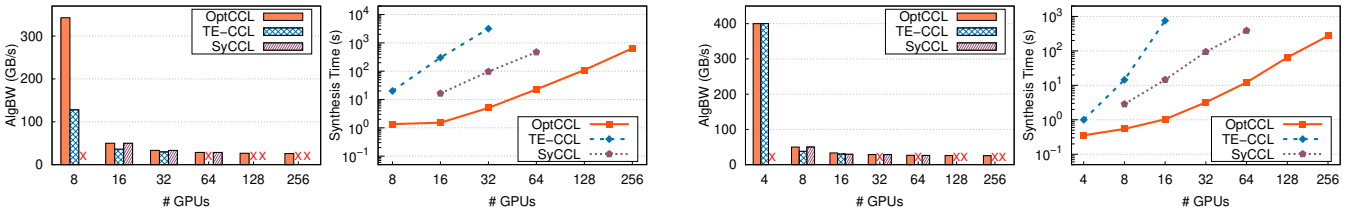


Figure 5: Performance in terms of algorithmic bandwidth and synthesis time for the A100 (left) and the DOE topologies (right) for the All-to-All collective. OptCCL synthesizes algorithms that have 1–1.3× and 1–1.3× better performance than TE-CCL and SyCCL, respectively. OptCCL also requires 2–500× lower synthesis time than TE-CCL and SyCCL.

the algorithms synthesized by TE-CCL are also far from optimal in most cases because it does not scale to inputs with a large number of chunks (performance improves with larger number of chunks). SyCCL significantly improves upon TE-CCL, both in terms of performance and synthesis time; however, it remains limited: it is unable to synthesize algorithms for more than 64–128 GPUs and often produces suboptimal algorithms. TACCL has scalability similar to SyCCL, but produces algorithms that have significantly worse performance than optimal even for moderate number of GPUs.

OptCCL consistently synthesizes better algorithms—1–5× better than TE-CCL, 1–1.6× better than SyCCL, and 1.2–18× better than TACCL. Moreover, beyond a small number of GPUs, OptCCL also significantly improves upon synthesis time; for example, for 64 or more GPUs, OptCCL synthesizes algorithms 3–32× faster than SyCCL and 5–60× faster than TACCL. Much of OptCCL's benefit in terms of synthesis time is due to the Mirrored Dantzig-Wolfe technique introduced in §6; see §A.11.

7.2 OptCCL for multiple concurrent collectives

For multiple collectives, we use an 8-host (64-GPU) A100 topology organized around 2 pods of 32 GPUs each. The setup uses 8 All-to-All collectives (one within each host), 16 ReduceScatter collectives (8 per pod, one for each rank within the pod), and 32 point-to-point communications (one for each pair of GPUs with the same rank across pods); the total data size for ReduceScatter and point-to-point communications is 2× the data size for the All-to-All collective. We evaluate the total time required to complete all the collectives.

Table 2 compares performance of various methods for synthesizing algorithms for multiple collectives. "Obvious" refers to first generating algorithms for individual collectives, and then executing them sequentially; this is the only approach we know for existing techniques to handle multiple concurrent collectives. "Overlaid" refers to solving each collective separately, then optimally scaling them so that the collectives run concurrently. And, "All-in-one" captures the result from optimizing for all collectives in a single formulation (§5.3). For each approach, we use OptCCL.

Table 2: Performance for multiple concurrent collectives on 8 host A100 topology (with 64 GPUs), normalized by corresponding all-in-one performance. Discussion in §7.2.

	With NVLinks	Without NVLinks
Obvious	6.25×	4.87×
Overlaid	1×	1.31×
All-in-one	1×	1×

With NVLinks, just overlaying the results already gains a 6× improvement in performance (thus, OptCCL performance for multiple concurrent collectives will be at least this much better than existing techniques), and solving the unified linear program from §5.3 gives no extra benefit. In this particular topology, we believe the overlaid solution is optimal because by optimizing the individual collectives for our secondary objective, we reduce contention in important links, which allows the overlaid solution to be better.

Without NVLinks, overlaying the results still yields more than 3× improvement; in addition, our all-in-one formulation gains an additional 1.3× improvement in performance.

8 Related work

We have already evaluated OptCCL against existing state-of-the-art techniques for synthesizing collective communication algorithms. We briefly discuss them, and other related works below.

Heuristics. The most widely-deployed systems for collective communication today [9, 10] use algorithms that are not optimized for the underlying host hardware and network topology, and are known to perform far from optimal. Several recent works [14, 21, 24, 26] present techniques to synthesize heuristics that empirically perform better than widely-deployed collective communication libraries;

however, as shown in our evaluation, even the state-of-the-art of these [21] can perform far from optimal and have poor scalability.

Optimality-scalability tussle. SCCL [3] uses an SMT solver to generate optimal algorithms. While the original SCCL paper did not focus on generality, it is not too hard to extend SCCL to model various hardware functionalities and concurrent collectives—after all, it uses an SMT solver that enables writing arbitrarily complex constraints. However, SMT solvers, without specialized optimizations, are notoriously slow. Unsurprisingly, many prior studies have already established that SCCL is unable to generate algorithms beyond a small number of GPUs.

The works closest to us are TE-CCL [15] and SyCCL [5]. The core similarity between TE-CCL and OptCCL is that both of them build upon the broad class of multicommodity-flow-based formulation. However, beyond that, they are fundamentally different. TE-CCL captures collective structure within the flow constraints, enforcing integer constraints on flow variables which eventually limits their scalability. TE-CCL, admitting its scalability limitations, also presents heuristics for inputs with more than a few GPUs (in which case, it loses the benefits of synthesizing optimal algorithms).

SyCCL [5] introduces new techniques to improve TE-CCL's scalability by exploiting symmetry in host hardware and network topologies. SyCCL defines symmetry with respect to the hierarchical structure of commonly used network topologies, and classifies GPUs into isomorphic groups with respect to sets of identical links. SyCCL enforces symmetry *within* the route of each piece of data: the pattern used to route data within one GPU group must be the same in every isomorphic group. Unfortunately, as briefly mentioned in the SyCCL paper (and as we formally establish in §A.12), enforcing the symmetry in such a manner can no longer guarantee optimality. Thus, SyCCL helps TE-CCL improve scalability but only by giving up on optimality.

OptCCL uses a fundamentally new formulation that enables it to break the optimality-scalability tussle; indeed, OptCCL always synthesizes optimal algorithms while maintaining generality and achieving improved scalability. OptCCL also uses a different indistinguishability-based definition of symmetry: via isomorphisms between labelings of the entire graph and using this symmetry to simplify the formulation while preserving optimality guarantees. Indeed, OptCCL techniques may be useful to improve TE-CCL's generality and scalability.

Scalable and optimal; but, give up on generality. A work concurrent to us—ForestColl [28]—introduces intriguing combinatorial techniques that enable synthesis of optimal collective communication algorithms for a subset of the collectives. By focusing on a certain set of collectives that can be modeled as data transmission over a collection of spanning trees, ForestColl is able to achieve scalability (in many cases, even better than OptCCL); see §A.13. Unfortunately, by exclusively relying on spanning trees, ForestColl gives up significantly in terms of generality. First, ForestColl is unable to synthesize collective communication algorithms for some of the collectives (*e.g.*, All-to-All): any spanning tree packing based technique only works on collectives that can be represented as spanning trees; All-to-All, and point-to-point communication in general cannot be represented in this way. Second, as evaluated in §A.13, ForestColl is unable to model shared bandwidth constraints in memory components without giving up on scalability and/or optimality. Finally, ForestColl is unable to generate algorithms when collectives do not span the entire set of GPUs in the topology (*e.g.*, in concurrent collective case)—this is again quite fundamental to the techniques used in ForestColl (packing of spanning trees): if some collective does not span all the GPUs, then their approach will require packing of so-called (directed) Steiner trees which is known to be a computationally hard problem, and does not even admit a good approximation [6].

9 Limitations and future directions

In this section, we outline several limitations of OptCCL and discuss interesting future directions that remain open.

Improving OptCCL generality. As discussed in §5, OptCCL formulation is general—it supports existing hardware functionalities, a broad definition of collectives, and supports optimizing completion time for multiple concurrent collectives. We outline several interesting directions to further improve OptCCL's generality. First, while we provide OptCCL implementation for all existing collectives, extending OptCCL to new collectives will require carefully defining usage variables for that collective. Second, our proofs and evaluation primarily focus on large message sizes where transmission delays dominate; it would be interesting to extend our proofs and evaluation to the case of small messages where propagation delays may dominate (*e.g.*, under the so-called α–β model). We believe this is fairly straightforward (using techniques similar to [15]): when all transmissions use fixed-size chunks, the time to send a chunk across a link is a fixed quantity; the propagation delay can thus be folded into an effective link bandwidth, after which the rest of OptCCL's techniques apply unchanged. Finally, the problem of synthesizing collective communication algorithms for multiple concurrent collectives that are optimal with respect to objectives beyond makespan (*e.g.*, flow-time or weighted completion time as in the coflow literature [1]) remains an intriguing open problem.

Real-world implications. OptCCL evaluation uses simulations. The natural next step here is to integrate OptCCL with existing collective communication libraries [9, 10], and validate OptCCL's performance benefits over real testbeds and over end-to-end ML training and inference applications.

10 Conclusion

OptCCL is a technique that, for any given host and network hardware and topology, synthesizes optimal collective communication algorithms. OptCCL is general: it enables synthesizing optimal algorithms for all existing collectives, for a large class of existing and future hardware, and for multiple concurrent collectives sharing host and network resources. Evaluation of OptCCL across multiple host and network hardware and topologies suggests that OptCCL is also scalable: it consistently synthesizes optimal collective communication algorithms for hundreds of GPUs within tens of minutes.

Acknowledgments

We would like to thank SIGCOMM reviewers for their insightful comments. This research was in part supported by NSF grant CNS-1704742. This work does not raise any ethical issues.

References

[1] Saksham Agarwal, Shijin Rajakrishnan, Akshay Narayan, Rachit Agarwal, David Shmoys, and Amin Vahdat. 2018. Sincronia: Near-optimal network design for coflows. In *SIGCOMM*.

[2] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. 1993. *Network flows - theory, algorithms and applications.* Prentice Hall, Upper Saddle River, NJ.

[3] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing optimal collective algorithms. In *PPoPP*.

[4] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: GPU-efficient communication scheduling for deep learning training. In *SIGCOMM*.

[5] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu, Yifan Yang, Yichi Xu, Zhilong Zheng, Yu Guan, Kun Qian, Ying Liu, Mingwei Xu, Tianshu Wang, Ning Wang, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2025. SyCCL: Exploiting symmetry for efficient collective communication scheduling. In *SIGCOMM*.

[6] Joseph Cheriyan and Mohammad R Salavatipour. 2006. Hardness and approximation results for packing Steiner trees. *Algorithmica* 45, 1 (2006), 21–43.

[7] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia A100 tensor core GPU: Performance and innovation. *IEEE Micro* (2021).

[8] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research* (2023).

[9] NVIDIA collective communications library (NCCL). 2026. https://developer.nvidia.com/nccl.

[10] ROCm communication collectives library (RCCL). 2025. https://github.com/ROCm/rccl.

[11] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for distributed training at Meta scale. In *SIGCOMM*.

[12] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *ATC*.

[13] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *NSDI*.

[14] Heehoon Kim, Junyeol Ryu, and Jaejin Lee. 2024. TCCL: Discovering better communication paths for PCIe GPU clusters. In *ASPLOS*.

[15] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. 2024. Rethinking machine learning collective communication as a multi-commodity flow problem. In *SIGCOMM*.

[16] C St JA Nash-Williams. 1961. Edge-disjoint spanning trees of finite graphs. *Journal of the London Mathematical Society* 1, 1 (1961), 445–450.

[17] NERSC. 2026. Perlmutter architecture. https://docs.nersc.gov/systems/perlmutter/architecture/.

[18] A.K. Parekh and R.G. Gallager. 1993. A generalized processor sharing approach to flow control in integrated services networks: the single-node case. *IEEE/ACM Transactions on Networking* (1993).

[19] A.K. Parekh and R.G. Gallager. 1994. A generalized processor sharing approach to flow control in integrated services networks: the multiple node case. *IEEE/ACM Transactions on Networking* (1994).

[20] Amedeo Sapio, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtarik. 2021. Scaling distributed machine learning with in-network aggregation. In *NSDI*.

[21] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding collective algorithm synthesis using communication sketches. In *NSDI*.

[22] William Thomas Tutte. 1961. On the problem of decomposing a graph into n connected factors. *Journal of the London Mathematical Society* 1, 1 (1961), 221–230.

[23] Midhul Vuppalapati, Saksham Agarwal, Henry Schuh, Baris Kasikci, Arvind Krishnamurthy, and Rachit Agarwal. 2024. Understanding the host network. In *SIGCOMM*.

[24] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. 2020. Blink: Fast and generic collectives for distributed ml. *Proceedings of Machine Learning and Systems* (2020).

[25] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. 2024. Rail-only: A low-cost high-performance network for training LLMs with trillion parameters . In *HOTI*.

[26] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. 2024. TACOS: Topology-aware collective algorithm synthesizer for distributed machine learning . In *MICRO*.

[27] Yongji Wu, Yechen Xu, Jingrong Chen, Zhaodong Wang, Ying Zhang, Matthew Lentz, and Danyang Zhuo. 2024. MCCS: A service-based approach to collective communication for multi-tenant cloud. In *SIGCOMM*.

[28] Liangyu Zhao, Saeed Maleki, Yuanhong Wang, Zezhou Wang, Ziyue Yang, Hossein Pourreza, and Arvind Krishnamurthy. 2026. ForestColl: Throughput-optimal collective communications on heterogeneous network fabrics. In *NSDI*.

A Deferred details

A.1 An MILP formulation for AllGather with multiple paths

The full mixed integer programming formulation for AllGather in which data can traverse multiple paths is provided below.

$$\min_{f,w} \quad \sum_{e\in E}\sum_{t=0}^{T-1} c_e \sum_{o\in V_o^T}\sum_{h\in[H]} w_{\kappa(e,t),o,h} \tag{17}$$

$$\text{s.t.} \quad \sum_{e\in\delta^-(v)} f_{e,o,d,h} - \sum_{e\in\delta^+(v)} f_{e,o,d,h} = 0$$
$$\text{for } o\in V_o^T, d\in V_d^T, v\in V^T\setminus\{o,d\}, h\in[H]$$
$$\sum_{e\in\delta^-(o)} f_{e,o,d,h} - \sum_{e\in\delta^+(o)} f_{e,o,d,h} = 1$$
$$\text{for } o\in V_o^T, d\in V_d^T, h\in[H]$$
$$\sum_{e\in\delta^-(d)} f_{e,o,d,h} - \sum_{e\in\delta^+(d)} f_{e,o,d,h} = -1$$
$$\text{for } o\in V_o^T, d\in V_d^T, h\in[H]$$
$$w_{e,o,h} \geq f_{e,o,d,h}$$
$$\text{for } e\in E^T, o\in V_o^T, d\in V_d^T, h\in[H]$$
$$\sum_{e\in L}\sum_{o\in V_o^T}\sum_{h\in[H]} w_{\kappa(e,t),o,h} \leq Hsb(L)$$
$$\text{for } L\in\mathcal{L}, t\in\{0,\dots,T-1\}$$
$$f\in\{0,1\}, w\geq 0.$$

Theorem 7. *MILP*(17) *is an exact formulation for AllGather with* H *uniform-sized chunks.*

The proof of this theorem is nearly identical to the proof of Theorem 1, which can be found in §A.2, so we do not include it here. We also note that by adjusting the net flow at origin and destination vertices for each chunk, we can easily extend this formulation to allow for non-uniform chunk sizes.

A.2 Proof of Theorem 1

We consider a mixed integer nonlinear program (MINLP) where the constraint (4) in MILP(6) is replaced with the constraint

$$w_{e,o} = \max_{d\in V_d^T} f_{e,o,d} \qquad \text{for } e\in E^T, o\in V_o^T. \tag{18}$$

That is, we consider the following MINLP:

$$\min_{f,w} \quad \sum_{e\in E}\sum_{t=0}^{T-1} c_e \sum_{o\in V_o^T} w_{\kappa(e,t),o}$$
$$\text{s.t.} \quad (1)-(3),(5),(18);\ f\in\{0,1\}, w\geq 0 \tag{19}$$

The proof below then establishes that this nonlinear formulation is exact, and that the MILP is equivalent to the MINLP. We will first prove the following theorem:

Theorem 8. *MINLP*(19) *is an exact formulation for Broadcast and AllGather.*

Proof. Define an algorithm to be minimal if no data transmissions can be removed from the algorithm without making it infeasible. We prove this theorem by showing that a minimal algorithm with step size s and number of steps T corresponds to a solution to (19), and argue that the correspondence is bidirectional.

We start with an algorithm for AllGather with step size s and T total steps. Recall that this algorithm specifies exactly what data is transferred along each link at each of the T steps. Since the data is indivisible in this model, we always send 1 unit of data at each step, and we can identify pieces of data by their origin. Say we send data originating at GPU o from i to j at step t. Then let $\hat{o} = (o, 0)$, and $\hat{e} = \kappa((i,j),t)$ be the corresponding edge in G^T. We set $w_{\hat{e},\hat{o}} = 1$. Since this is a valid algorithm, the bandwidth on any set of links L is no more than $b(L)$ at each step. This means that no more than $sb(L)$ data is sent on edges in L each step. Summing over all data identified by their origins gives exactly the constraint (5).

Now, we just need to find feasible f values. Since the algorithm is correct, data originating at o must be routed to every destination d. For every origin-destination pair (o,d) and edge $\hat{e}\in E^T$, set the corresponding $f_{\hat{e},\hat{o},\hat{d}}$ variable to 1 if the data that eventually arrived at d was transmitted along a link corresponding to (i,j) at time t, and zero otherwise. (If the data remained in memory on component v for step t of the algorithm, we set $f_{\kappa((v,v),t),\hat{o},\hat{d}}$ to 1.)

This assignment of f variables clearly satisfies the flow constraints: at the destination, there are only in-edges, and one of them has value 1. The analogous statement is true for the origin. And for all other intermediate vertices, the data leaves some component if and only if it entered the component at a previous time. It also satisfies constraint (18): $w_{\hat{e},\hat{o}} = 1$ when some data is transmitted along the edge at step t. By the minimality of the algorithm, this can only occur if this data eventually arrives at some destination d, and so $f_{\hat{e},\hat{o},\hat{d}} = 1$. And if $w_{\hat{e},\hat{o}} = 0$, we trivially have that $f_{\hat{e},\hat{o},\hat{d}} = 0$ for all $\hat{d}$.

For the other direction, start with a solution to MINLP(19). We use the w values to determine exactly what data to send at each step: if $w_{\hat{e},\hat{o}} = 1$, where $\hat{e} = ((i,t),(j,t+1))$, we transmit data from component i to component j at step t. This is a correct algorithm: The bandwidth constraint requires the data transmitted to not exceed the bandwidth of any link. And the flow constraints ensure that the data arrives at all GPUs.

Finally, since we have established that the w variables exactly capture the amount of data transmitted between components at each step, the objectives also match. □

Proof of Theorem 1. We argue a correspondence between optimal solutions to MILP(6) and optimal solutions to MINLP(19).

Given a solution f, w to MINLP(19), let $f'_{e,o,d} = f_{e,o,d}$ for all $e\in E^T$, $o\in V_o^T$, and $d\in V_d^T$, and $w'_{e,o} = w_{e,o}$ for all e and o. Then f', w' is a feasible solution to MILP(6) with the same objective value, as the only difference is constraint (4), which still holds since $w'_{e,o} = \max_{d\in V_d^T} f'_{e,o,d}$. For the other direction, given an optimal solution f', w' to MILP(6), similarly let $f_{e,o,d} = f'_{e,o,d}$ for all $e\in E^T$, $o\in V_o^T$, and $d\in V_d^T$, and set $w_{e,o} = \max_{d\in V_d^T} f_{e,o,d}$ for all e and o. Then f, w is a feasible solution to MINLP(19) because the flow constraints are identical, (18) is true by construction, and (5) holds because $w_{e,o} \leq w'_{e,o}$ for all e and o. In fact, $w_{e,o} = w'_{e,o}$ for all e where $c_{\mu(e)} > 0$, because otherwise we could have reduced the value of $w'_{e,o}$ to get a better objective for MILP(6), violating our optimality assumption.

Therefore, the objective value $\sum_{e\in E}\sum_{t=0}^{T-1} c_e \sum_{o\in V_o^T} w_{\kappa(e,t),o}$ is the same. □

A.3 Proof of Theorem 2

We begin with the simpler direction: taking a solution (f, w) to LP(7)$[s, T]$ and constructing a solution to LP(9)$[sT, T]$ with identical objective value. This is straightforward; we claim that (f, w) is also a feasible solution for LP(9)$[sT, T]$. Note that both formulations have the same objective and constraints (1) – (4). And constraint (5) implies constraint (8) because $T \geq 1$.

We will start by outlining the argument for the more complicated direction before proving it rigorously. We want to argue that we can take a solution (f, w) to LP(9)$[R, T]$ and use it to construct a solution to LP(7)$[s', T']$ for some choice of s' and T'. Our general idea will be to divide the data into small chunks, then route each chunk according to (f, w), staggering the transmissions so that each chunk is transmitted at a different time. This will allow us to argue that since our constructed solution obeys the bandwidth constraints on average, it also obeys them at each step.

First, we choose an integer K according to $K > R(T-1)/\epsilon$, and also set $s' = R/K$. We begin our construction by dividing the data into K identically sized chunks, though these chunks need not be indivisible. We then begin routing one chunk at every step for the first K steps, where the route of each chunk is determined by (f, w). In particular, if (f, w) indicates that we should transmit x units of data along edge e at time t, in our new solution, for each chunk $k \in [K]$, we will transmit x/K units of data along edge e at time $t + k - 1$.

Using this strategy, we can route all our data in $T' = T + K - 1$ steps. All that remains is to argue that this solution is feasible for LP(7)$[s', T']$. Since our costs are not time-dependent, our objective remains the same. We can also check that constraints (1) – (4) hold. For constraint (5), we can argue that the total usage on any edge is bounded by the total usage over all T steps of the solution to LP(9)$[R, T]$. Therefore, we get that the usage at each step of our constructed solution is bounded by $Rb(L)/K$, which simplifies to our desired bound of $s'b(L)$ using our definition of s'. And this solution has completion time $s'T' = \frac{R(T+K-1)}{K} < R + \epsilon$.

We now make the above sketch rigorous. We will introduce some new notation. In a time-expanded network G^T, let E^t be the set of edges that originate at time t. Specifically, $E^t = \{((i, t'), (j, t'+1)) \in E^T : t' = t\}$.

Recall K, $s' = R/K$, and $T' = T + K - 1$ as fixed above. We will describe how to use a solution (f, w) to LP(9)$[R, T]$ to construct a feasible solution to program LP(7)$[s', T']$. In particular, we will describe values (f', w') that satisfy this LP.

Let $\hat{e} \in E^{T'}$ and $t' = \tau(\hat{e})$. Then for every $\hat{e}$, $o \in V_o^{T'}$ and $d \in V_d^{T'}$,

$$f'_{\hat{e},o,d} = \begin{cases} \sum_{t=t'-K+1}^{t'} \frac{f_{\kappa(\mu(\hat{e}),t),o,d}}{K} + \frac{K-1-t'}{K} \\ \qquad \text{if } t' \leq K-1 \text{ and } \mu(\hat{e}) = (\mu(o), \mu(o)) \\ \sum_{t=t'-K+1}^{t'} \frac{f_{\kappa(\mu(\hat{e}),t),o,d}}{K} + \frac{t'-T+1}{K} \\ \qquad \text{if } t' \geq T \text{ and } \mu(\hat{e}) = (\mu(d), \mu(d)) \\ \sum_{t=t'-K+1}^{t'} \frac{f_{\kappa(\mu(\hat{e}),t),o,d}}{K} \\ \qquad \text{otherwise} \end{cases}$$

where $f_{\kappa(\mu(\hat{e}),t),o,d} = 0$ if $t < 0$ or $t \geq T$.

Similarly, we define

$$w'_{\hat{e},o} = \begin{cases} \sum_{t=t'-K+1}^{t'} \frac{w_{\kappa(\mu(\hat{e}),t),o}}{K} + \frac{K-1-t'}{K} \\ \qquad \text{if } t' \leq K-1 \text{ and } \mu(\hat{e}) = (\mu(o), \mu(o)) \\ \sum_{t=t'-K+1}^{t'} \frac{w_{\kappa(\mu(\hat{e}),t),o}}{K} + \frac{t'-T+1}{K} \\ \qquad \text{if } t' \geq T \text{ and } \mu(\hat{e}) = (\mu(d), \mu(d)) \text{ for } d \in V_d^T \\ \sum_{t=t'-K+1}^{t'} \frac{w_{\kappa(\mu(\hat{e}),t),o}}{K} \\ \qquad \text{otherwise} \end{cases}$$

where again, $w_{\kappa(\mu(\hat{e}),t),o} = 0$ if $t < 0$ or $t \geq T$.

These expressions come from overlaying the K staggered solutions (f, w), where each one is only on $1/K$ of the data. The extra terms in the first two cases capture the edge-cases where some data has yet to leave its origin, or some data has already arrived at its destination.

We first argue that (f', w') is feasible for all the constraints in LP(7)$[s', T']$.

For given $o' \in V_o^{T'}, d' \in V_d^{T'}$, first define $o = (\mu(o'), 0) \in V_o^T$ and $d = (\mu(d'), T) \in V_d^T$. Then for any $v \in V^{T'}$, where $\mu(v)$ is neither $\mu(o')$ nor $\mu(d')$,

$$\begin{aligned} &\sum_{\hat{e}\in\delta^-(v)} f'_{\hat{e},o',d'} - \sum_{\hat{e}\in\delta^+(v)} f'_{\hat{e},o',d'} \\ &= \sum_{\hat{e}\in\delta^-(v)} \sum_{t=\tau(\hat{e})-K+1}^{\tau(\hat{e})} \frac{f_{\kappa(\mu(\hat{e}),t),o,d}}{K} - \sum_{\hat{e}\in\delta^+(v)} \sum_{t=\tau(\hat{e})-K+1}^{\tau(\hat{e})} \frac{f_{\kappa(\mu(\hat{e}),t),o,d}}{K} \\ &= \sum_{e\in\delta^-(\mu(v))} \sum_{t=\tau(v)-K}^{\tau(v)-1} \frac{f_{\kappa(e,t),o,d}}{K} - \sum_{e\in\delta^+(\mu(v))} \sum_{t=\tau(v)-K+1}^{\tau(v)} \frac{f_{\kappa(e,t),o,d}}{K} \\ &= \sum_{t=\tau(v)-K+1}^{\tau(v)} \left(\sum_{e\in\delta^-(\mu(v))} \frac{f_{\kappa(e,t-1),o,d}}{K} - \sum_{e\in\delta^+(\mu(v))} \frac{f_{\kappa(e,t),o,d}}{K} \right) \\ &= \sum_{t=\tau(v)-K+1}^{\tau(v)} \left(\sum_{\hat{e}\in\delta^-((\mu(v),t))} \frac{f_{\hat{e},o,d}}{K} - \sum_{\hat{e}\in\delta^+((\mu(v),t))} \frac{f_{\hat{e},o,d}}{K} \right). \end{aligned}$$

Note in the second equality, the e under the sum is in G, not $G^{T'}$.

Now, since $\mu(v)$ is neither $\mu(o')$ nor $\mu(d')$, by constraint (1) in LP(9)$[R, T]$, the expression evaluates to zero, so constraint (1) in LP(7)$[s', T']$ holds.

The other cases are slightly more complicated. Say $\mu(v) = \mu(o')$. If $\tau(v) \geq K$, the expression still evaluates to zero. Otherwise, if $0 < \tau(v) \leq K-2$, there is an additional $\frac{K-\tau(v)}{K}$ in-flow term from one edge, and a $\frac{K-1-\tau(v)}{K}$ out-flow term. This gives us:

$$\begin{aligned} &\sum_{\hat{e}\in\delta^-(v)} f'_{\hat{e},o',d'} - \sum_{\hat{e}\in\delta^+(v)} f'_{\hat{e},o',d'} \\ &= \sum_{t=\tau(v)-K+1}^{\tau(v)} \left(\sum_{\hat{e}\in\delta^-((\mu(v),t))} \frac{f_{\hat{e},o,d}}{K} - \sum_{\hat{e}\in\delta^+((\mu(v),t))} \frac{f_{\hat{e},o,d}}{K} \right) - \frac{1}{K} \end{aligned}$$

For all values of t except $t = 0$, the summand evaluates to zero by constraint (1) in LP(9)$[R, T]$. When $t = 0$, by constraint (2) in LP(9)$[R, T]$, the summand evaluates to $\frac{1}{K}$. So constraint (1) in LP(7)$[s', T']$ still holds.

And if $\tau(v) = 0$, the same summand evaluates to $\frac{1}{K}$ for exactly one choice of t, but the extra term in this case is a positive $\frac{K-1}{K}$ instead of $-\frac{1}{K}$. So constraint (2) in LP(7)$[s', T']$ also holds.

A symmetric argument for the cases where $\mu(v) = \mu(d')$ shows that constraint (3) in LP(7)$[s', T']$ also holds.

We need to be slightly more careful in the case where $\mu(o') = \mu(d')$, as the claim that the sum evaluates to zero if $K \le \tau(v)$ is not true, but the argument is fundamentally the same.

For constraint (4) in LP(7), we have the following, given $\hat{e}$ falls into the "otherwise" case for the definitions of f' and w'.

$$\begin{aligned} w'_{\hat{e},o'} &= \sum_{t=\tau(\hat{e})-K+1}^{\tau(\hat{e})} \frac{w_{\kappa(\mu(\hat{e}),t),o}}{K} \\ &\ge \sum_{t=\tau(\hat{e})-K+1}^{\tau(\hat{e})} \frac{\max_d f_{\kappa(\mu(\hat{e}),t),o,d}}{K} \\ &\ge \max_d \sum_{t=\tau(\hat{e})-K+1}^{\tau(\hat{e})} \frac{f_{\kappa(\mu(\hat{e}),t),o,d}}{K} = \max_d f'_{\hat{e},o,d} \end{aligned}$$

In the other two cases, the extra additive term is the same for both w' and f'. So the constraint also holds.

For constraint (5) in LP(7), we find for any $L \in \mathcal{L}$ and any $t' \in \{0, \dots, T'-1\}$,

$$\begin{aligned} \sum_{e\in L} \sum_{o'\in V_o^{T'}} w'_{\kappa(e,t'),o'} &= \sum_{e\in L} \sum_{o\in V_o^T} \sum_{t=t'-K+1}^{t'} \frac{w_{\kappa(e,t),o}}{K} \\ &\le \sum_{e\in L} \sum_{o\in V_o^T} \sum_{t=0}^{T-1} \frac{w_{\kappa(e,t),o}}{K} \\ &\le \frac{Rb(L)}{K} = s'b(L), \end{aligned}$$

where we are using the fact that we have no bandwidth limitations on self-loops.

The last task is to show that our conversion preserves objective cost. We can write:

$$\begin{aligned} \sum_{e\in E} \sum_{t'=0}^{T'-1} c_e \sum_{o'\in V_o^{T'}} w'_{\kappa(e,t'),o'} &= \sum_{e\in E} c_e \sum_{t'=0}^{T+K-2} \sum_{o\in V_o^T} \sum_{t=t'-K+1}^{t'} \frac{w_{\kappa(e,t),o}}{K} \\ &= \sum_{e\in E} c_e \sum_{o\in V_o^T} \sum_{t=0}^{T-1} \sum_{t'=t}^{t+K-1} \frac{w_{\kappa(e,t),o}}{K} \\ &= \sum_{e\in E} c_e \sum_{o\in V_o^T} \sum_{t=0}^{T-1} K \frac{w_{\kappa(e,t),o}}{K} \\ &= \sum_{e\in E} \sum_{t=0}^{T-1} \sum_{o\in V_o^T} c_e w_{\kappa(e,t),o}. \end{aligned}$$

Notice that we need not worry about the other cases in the definition for w' because those edges are guaranteed to have zero cost.

A.4 Computing R

In this section, we briefly discuss how we can compute R for use with LP(9).

As discussed, we adapt a strategy from ForestColl that computes throughput by solving many flow problems [28]. However, instead of doing binary search to find the optimal throughput, we solve a linear program to find the bottleneck value for each constructed flow problem:

Construct a graph G' from G by adding a source vertex s and an arc (s, g) for every GPU $g \in V_d$. Then for every GPU $g \in V_d$, solve the following LP:

$$\begin{aligned} \max \quad & x \\ \text{s.t.} \quad & \sum_{e\in\delta^+(s)} f_e = |V_d| \cdot x \\ & \sum_{e\in\delta^-(g)} f_e = |V_d| \cdot x \\ & \sum_{e\in\delta^-(v)} f_e = \sum_{e\in\delta^+(v)} f_e && \text{for } v \in V' \setminus \{s, g\} \\ & \sum_{e\in L} f_e \le b(L) && \text{for } L \in \mathcal{L} \\ & f_{(s,h)} \le x && \text{for } h \in V_d \\ & x, f \ge 0. \end{aligned}$$

Let $x^*(g)$ denote the optimal value for GPU g. Then the best choice of R is given by $R = 1/(\min_{g\in V_d} x^*(g))$.

A.5 Proof of Theorem 3

We first prove a lemma about ideal topologies.

Lemma 9. *Given an ideal topology, the solution produced by LP(9) is algorithm-feasible.*

Proof. Our f variables indicate the path that all pieces of data should follow, and the flow constraints perfectly capture that enough data arrives at every destination. So to determine the feasibility of an algorithm, we need only check that our usage variables correctly reflect how much data actually travels along each link.

Note that when we first introduced usage variables in MILP(6), this was not a problem. But by subdividing the data, we cannot guarantee that the pieces of data being aggregated together actually can be combined. Indeed, we should not expect that this is the case, or else we would have found a polynomial-time algorithm for the Steiner tree problem.

The only way for a usage variable to have an incorrect value is if we calculate it by taking the maximum of flows that should not be combined. For this to occur, we must have disjoint segments of data from the same origin, each with a different destination, all using the same link.

This cannot happen on the last link, because then they could not have different destinations. If something like this happens on the first link, we can exchange segments of data for an equivalent solution in which the data segments are not disjoint on any first links. (We exchange the entire path of two equivalently sized segments of data, so that all downstream flow constraints and copy operations are preserved.) In fact, disjoint data can only occur if the data first diverged before reconvening on some link, which would require such a link to be at least the third link. So we have already shown our result if no piece of data travels more than three layers.

In the four-layer case, our argument must focus on the third layer of edges. With our additional restriction on which edges have

bandwidth constraints, we can ignore any demands that remain in the same host: since there are no costs or bandwidth constraints, increasing the w value to reflect the true usage causes no problems. It is only a problem if this happens on an inter-host link. But in this case, we can also exchange the segments of data that come before without impacting the w variables on the inter-host links. □

Together with Theorem 2, this proves Theorem 3.

A.6 An exponentially sized formulation

LP(9) is a relaxation is because we cannot guarantee that flows can always combine if they share the same origin. If everything works as we desire, then letting w be the maximum over all flows f from the same origin should accurately describe how much data we actually need to send on an edge. But this may not always be the case. For example, if two flows f from the same origin share an edge but represent disjoint pieces of data, the usage on that edge is not described by the maximum of the flows, but rather their sum. To remedy this, we will describe an exponentially sized linear program that will prevent flows from combining unless they share their entire path from their origin.

To do this, we will modify our time-expanded network to have $2^{|V_d|} - 1$ copies, one for every non-empty subset of V_d. And we will modify our edge set so that if $(v_1, v_2) \in E$, then our time-expanded network will contain edges from (v_1, t, S_1) to $(v_2, t+1, S_2)$ for all subsets $\emptyset \neq S_1 \subseteq S_2 \subseteq V_d$. Then we formulate a nearly-standard multicommodity flow problem on this modified graph, with joint capacities. The formulation has objective:

$$\min \sum_{(v_1,v_2)=e\in E} \sum_{t=0}^{T-1} \sum_{S_1,S_2:S_1\subseteq S_2\subseteq V_d} c_e \sum_{o\in V_o^T} f_{((v_1,t,S_1),(v_2,t+1,S_2)),o}.$$

We have flow constraints where we define origin vertices of the form $(v, 0, V_d)$ for $v \in V_o$ and destination vertices of the form $(v, T, \{v\})$ for $v \in V_d$. We also require an analogous bandwidth constraint:

$$\sum_{t=0}^{T-1} \sum_{(v_1,v_2)\in L} \sum_{S_1,S_2:S_1\subseteq S_2\subseteq V_d} \sum_{o\in V_o^T} f_{((v_1,t,S_1),(v_2,t+1,S_2)),o} \leq Rb(L)$$

for $L \in \mathcal{L}$.

We can show that this formulation captures exactly algorithm-feasible solutions to our model by noticing that a vertex corresponding to subset S is only allowed to have outgoing flow that eventually reaches exactly the terminals in S.

A.7 Uniform chunk sizes

Translating the solution from LP(9) into a practical schedule executable by NCCL and similar communication libraries requires addressing two challenges. First, the LP specifies only the volume of data to transmit on each edge at each time, not the explicit assignment of data items to transmissions. Second, the LP solution permits arbitrary fractional data volumes, which are incompatible with the uniform chunk sizes required by most communication libraries.

To address the first concern, given the solution to our linear program, we can first perform a path decomposition to produce a set of paths for each origin-destination pair, then further aggregate these paths into trees. Now it is straightforward to interpret these trees on our time-expanded network as the communication pattern of a single piece of data.

The next challenge is to use these trees to construct a step-based solution that transmits data in chunks of some given chunk size. Our approach is an adaptation of weighted fair queueing [18, 19], originally described in the context of generalized processor sharing.

First we choose an integer parameter K, which indicates how many time steps we divide our fractional solution into. As we will see in the upcoming theorem statement, K controls a tradeoff: larger values of K track the fractional solution more closely, but make each time step shorter and therefore make the rounding of per-step edge capacities relatively more expensive.

The procedure takes as inputs a set of trees, where each tree τ has an associated value x_τ indicating how much data should be transmitted along that tree, along with a desired chunk size C. The goal is to transmit an average of $x_\tau/(KC)$ chunks of data at every step for every tree.

In order to prevent bursts of chunks sent along a shared edge, for each edge e in our underlying graph, we will maintain a queue of chunks waiting to be transmitted along e. We also define $r_e = \sum_{\tau:e\in\tau} x_\tau$ to be the (fractional) number of chunks that need to be transmitted on edge e.

Then, we perform the following procedure:

(1) Determine when chunks should be introduced into the system. For each tree τ, at time step t, we should add $\lfloor \frac{x_\tau}{KC}(t+1)\rfloor - \lfloor \frac{x_\tau}{KC}(t)\rfloor$ chunks into the system. (Note that this amount is always either $\lfloor \frac{x_\tau}{KC}\rfloor$ or $\lceil \frac{x_\tau}{KC}\rceil$.) Stop after x_τ/C chunks have been added.
(2) Following the standard weighted fair queueing algorithm, when the n-th chunk of tree τ arrives at some edge e at time step t, we calculate its virtual finish time for that edge as
$$V(n) = \max(t, V(n-1)) + \frac{KCr_e}{x_\tau \lceil r_e\rceil},$$
with $V(0) = 0$. $V(n)$ represents the time when the chunk would finish transmitting across e in our ideal fractional setting.
(3) For each edge e, at each time step, among all queued chunks from all trees, transmit the $\lceil r_e\rceil$ chunks with the smallest virtual finish time, breaking ties arbitrarily.
(4) After being transmitted along e at time step t, a chunk enters the queue for the next edge(s) at time step $t+1$. Importantly, the arriving chunk enters the queue of all child edges in its tree.

Theorem 10. *The transmission schedule generated by the above procedure has collective completion time at most*

$$R\left(1 + \frac{KC}{Rb_{\min}}\right)\left(1 + \frac{CT}{x_\tau} + \frac{2T}{K}\right)$$

for every tree τ, where $b_{\min}$ is the minimum bandwidth in our topology.

This theorem essentially says that for realistic topologies where $Rb_{\min}$ is very large in comparison to C, if we choose K appropriately, and x_τ is large enough for all trees we consider, then the completion time (and therefore the algorithmic bandwidth) of our resulting algorithm is very close to that of our optimal solution.

The bound of Theorem 10 depends on K through two terms with opposite effects. Theorem 4 follows by choosing a K that balances the two, which yields a looser but more interpretable bound.

Proof of Theorem 4. Let $x_{\min} = \min_\tau x_\tau$ denote the smallest data volume assigned to any tree in the decomposition. The bound of Theorem 10 is decreasing in x_τ, so applying it to the tree attaining $x_\tau = x_{\min}$ bounds the completion time of the entire schedule.

It remains to choose K. Equating the two K-dependent terms,

$$\frac{KC}{Rb_{\min}} = \frac{2T}{K} \quad \Longleftrightarrow \quad K = K^\star := \sqrt{\frac{2TRb_{\min}}{C}},$$

at which point both terms take the common (dimensionless) value

$$\gamma = \frac{K^\star C}{Rb_{\min}} = \frac{2T}{K^\star} = \sqrt{\frac{2CT}{Rb_{\min}}}.$$

Substituting $K = K^\star$ into Theorem 10 gives completion time at most

$$R\,(1+\gamma)\left(1+\gamma+\frac{CT}{x_{\min}}\right),$$

as claimed. □

Before we prove Theorem 10, we will prove the following lemma.

Lemma 11. *The transmission schedule generated by the above procedure satisfies the following properties:*

1. *The number of chunks transmitted on edge e at any time step is at most $\lceil r_e \rceil$.*
2. *All chunks of data are routed to all desired destinations.*
3. *The last chunk of tree τ reaches its destinations by step $K + TKC/x_\tau + 2T$.*

Proof. The first two properties are true by construction. So we will focus on the third property.

Fix a tree τ and a root-to-leaf path $e_1, e_2, \ldots, e_T$. We now track the n-th chunk of tree τ on this path.

Now we introduce some notation. Let $s(n)$ be the time step at which chunk n enters the system, let $t_i(n)$ be the arrival time of chunk n to edge e_i (the time step when the chunk enters the queue), let $V_i(n)$ be the virtual finish time at edge e_i, and let $P_i(n)$ be the actual transmission time along edge e_i according to our procedure.

Claim. For all $i \geq 1$ and $n \geq 1$, three inequalities hold:

$$V_i(n) \leq \frac{(n+i)KC}{x_\tau} + 2(i-1),$$
$$P_i(n) \leq \frac{(n+i)KC}{x_\tau} + 2i - 1,$$
$$t_{i+1}(n) \leq \frac{(n+i)KC}{x_\tau} + 2i.$$

We prove the claim by induction, first on i, then on n.

For the base case, when $i = 1$, $t_1(n) = s(n) \leq \frac{nKC}{x_\tau}$. Then we use induction on n to show $V_1(n) \leq \frac{(n+1)KC}{x_\tau}$.

$$V_1(1) \leq \frac{KC}{x_\tau} + \frac{KCr_e}{x_\tau \lceil r_e \rceil} \leq \frac{KC}{x_\tau} + \frac{KC}{x_\tau} = \frac{2KC}{x_\tau}$$

for the case when $n = 1$, and assuming the bound on V_1 holds for $n-1$, we get

$$V_1(n) \leq \max\left(\frac{nKC}{x_\tau}, \frac{nKC}{x_\tau}\right) + \frac{KCr_e}{x_\tau \lceil r_e \rceil} \leq \frac{nKC}{x_\tau} + \frac{KC}{x_\tau} = \frac{(n+1)KC}{x_\tau}.$$

By the bound from Parekh and Gallager [18, Theorem 1], we also get $P_1(n) \leq \frac{(n+1)KC}{x_\tau} + 1$ and $t_2(n) \leq \frac{(n+1)KC}{x_\tau} + 2$.

Now for the inductive step on i, we suppose $t_i(n) \leq \frac{(n+i-1)KC}{x_\tau} + 2(i-1)$. And we show that $V_i(n) \leq \frac{(n+i)KC}{x_\tau} + 2(i-1)$ by induction on n.

At $n = 1$,

$$V_i(1) \leq t_i(1) + \frac{KC}{x_\tau} \leq \frac{iKC}{x_\tau} + 2(i-1) + \frac{KC}{x_\tau} = \frac{(i+1)KC}{x_\tau} + 2(i-1).$$

Assuming the inner inductive hypothesis for $n-1$, we have that $V_i(n-1) \leq \frac{(n+i-1)KC}{x_\tau} + 2(i-1)$, the same bound as $t_i(n)$. So

$$V_i(n) \leq \frac{(n+i-1)KC}{x_\tau} + 2(i-1) + \frac{KC}{x_\tau} = \frac{(n+i)KC}{x_\tau} + 2(i-1).$$

And using the same bound as above, we get $P_i(n) \leq \frac{(n+i)KC}{x_\tau} + 2i - 1$ and $t_{i+1}(n) \leq \frac{(n+i)KC}{x_\tau} + 2i$, completing the proof of the claim.

The last chunk arrives at the leaf after time step

$$P_T\left(\frac{x_\tau}{C}\right) + 1 \leq \frac{(x_\tau/C + T)KC}{x_\tau} + 2T = K + \frac{TKC}{x_\tau} + 2T,$$

as desired. □

Proof of Theorem 10. The generated algorithm has two factors increasing the completion time. There is an additive term from the additional time steps needed to account for possible delays and there is a multiplicative factor to address potential bandwidth violations.

For the first, Lemma 11 states that the number of time steps required exceeds K by $\frac{TKC}{x_\tau} + 2T$.

Second, we cannot guarantee that for any edge e, the number of chunks that may be sent in one step $\frac{b(e)R}{KC}$ is an integer. This means that $\lceil r_e \rceil$ may actually exceed the number of chunks we are allowed to transmit on e in a single time step, violating our bandwidth constraints. To handle this, we can stretch our time steps so that they are $\frac{Rb_{\min}/(KC)+1}{Rb_{\min}/(KC)} = 1 + \frac{KC}{Rb_{\min}}$ times longer, which will ensure that no bandwidth constraint is violated.

So our new step size is $\frac{R}{K}\left(1 + \frac{KC}{Rb_{\min}}\right)$. Multiplying this by the number of time steps yields the bound in the theorem statement. □

A.8 Duplicating vertices

We present here a precise description of the modified time-expanded network used to model vertices that do not allow copies.

We modify our time-expanded network G^T so that for every $v \in \overline{V_c}$ and every step t, we make $|V_c|$ vertices, one associated with each vertex in V_c. We can denote these (v, t, u) for $u \in V_c$. Now, we describe how we add edges in this adjusted time-expanded network. For edges $\{(v_1, v_2) : v_1, v_2 \in V_c\}$ between vertices in V_c, we add edges from (v_1, t) to $(v_2, t+1)$ as before. For edges $\{(v_1, v_2) : v_1 \in V_c, v_2 \in \overline{V_c}\}$ to vertices in $\overline{V_c}$, we add edges from (v_1, t) to $(v_2, t+1, u)$ for each $u \in V_c$. For edges $\{(v_1, v_2) : v_1, v_2 \in \overline{V_c}\}$ between vertices in $\overline{V_c}$, we add edges (v_1, t, u) to $(v_2, t+1, u)$ for each $u \in V_c$. And finally, for edges $\{(v_1, v_2) : v_1 \in \overline{V_c}, v_2 \in V_c\}$ to vertices in V_c, we only add edges (v_1, t, v_2) to $(v_2, t+1)$.

A.9 Formulation and optimality for AllReduce

As described in §5.2, we obtain a formulation for AllReduce by concatenating the formulations for ReduceScatter and AllGather. We use a superscript (1) to denote variables and sets belonging to the ReduceScatter sub-formulation, and a superscript (2) for the AllGather sub-formulation; for instance, $T^{(1)}$ and $T^{(2)}$ are the time horizons (number of steps) of the time-expanded networks $G^{T^{(1)}}$ and $G^{T^{(2)}}$ used for ReduceScatter and AllGather, respectively.

$$\min_{f,w} \sum_{e\in E} c_e \left(\sum_{t_1=0}^{T^{(1)}-1} \sum_{d\in V_d^{T^{(1)}}} w^{(1)}_{\kappa(e,t_1),d} + \sum_{t_2=0}^{T^{(2)}-1} \sum_{o\in V_o^{T^{(2)}}} w^{(2)}_{\kappa(e,t_2),o} \right)$$

$$\text{s.t.} \quad \sum_{e\in\delta^-(v)} f^{(1)}_{e,o,d} - \sum_{e\in\delta^+(v)} f^{(1)}_{e,o,d} = 0 \qquad \text{for } o \in V_o^{T^{(1)}}, d \in V_d^{T^{(1)}}, v \in V^{T^{(1)}} \setminus \{o, d\}$$

$$\sum_{e\in\delta^-(o)} f^{(1)}_{e,o,d} - \sum_{e\in\delta^+(o)} f^{(1)}_{e,o,d} = z_{\mu(d)} \qquad \text{for } o \in V_o^{T^{(1)}}, d \in V_d^{T^{(1)}}$$

$$\sum_{e\in\delta^-(d)} f^{(1)}_{e,o,d} - \sum_{e\in\delta^+(d)} f^{(1)}_{e,o,d} = -z_{\mu(d)} \qquad \text{for } o \in V_o^{T^{(1)}}, d \in V_d^{T^{(1)}}$$

$$w^{(1)}_{e,d} \geq f^{(1)}_{e,o,d} \qquad \text{for } e \in E^{T^{(1)}}, o \in V_o^{T^{(1)}}, d \in V_d^{T^{(1)}}$$

$$\sum_{e\in\delta^-(v)} w^{(1)}_{e,d} - \sum_{e\in\delta^+(v)} w^{(1)}_{e,d} = 0 \text{ for } d \in V_d^{T^{(1)}}, v \in \overline{V_r}^{T^{(1)}}$$

$$\sum_{e\in\delta^-(v)} f^{(2)}_{e,o,d} - \sum_{e\in\delta^+(v)} f^{(2)}_{e,o,d} = 0 \qquad \text{for } o \in V_o^{T^{(2)}}, d \in V_d^{T^{(2)}}, v \in V^{T^{(2)}} \setminus \{o, d\}$$

$$\sum_{e\in\delta^-(o)} f^{(2)}_{e,o,d} - \sum_{e\in\delta^+(o)} f^{(2)}_{e,o,d} = z_{\mu(o)} \qquad \text{for } o \in V_o^{T^{(2)}}, d \in V_d^{T^{(2)}}$$

$$\sum_{e\in\delta^-(d)} f^{(2)}_{e,o,d} - \sum_{e\in\delta^+(d)} f^{(2)}_{e,o,d} = -z_{\mu(o)} \qquad \text{for } o \in V_o^{T^{(2)}}, d \in V_d^{T^{(2)}}$$

$$w^{(2)}_{e,o} \geq f^{(2)}_{e,o,d} \qquad \text{for } e \in E^{T^{(2)}}, o \in V_o^{T^{(2)}}, d \in V_d^{T^{(2)}}$$

$$\sum_{e\in\delta^-(v)} w^{(2)}_{e,o} - \sum_{e\in\delta^+(v)} w^{(2)}_{e,o} = 0 \text{ for } o \in V_o^{T^{(2)}}, v \in \overline{V_c}^{T^{(2)}}$$

$$\sum_{t_1=0}^{T^{(1)}-1} \sum_{e\in L} \sum_{d\in V_d^{T^{(1)}}} w^{(1)}_{\kappa(e,t_1),d} + \sum_{t_2=0}^{T^{(2)}-1} \sum_{e\in L} \sum_{o\in V_o^{T^{(2)}}} w^{(2)}_{\kappa(e,t_2),o} \leq Rb(L) \qquad \text{for } L \in \mathcal{L}$$

$$\sum_{d\in V_d} z_d = |V_d|$$

$$f^{(1)}, f^{(2)}, w^{(1)}, w^{(2)}, z \geq 0 \tag{20}$$

Intuitively, the variable z selects the *intermediate state* of the AllReduce: for each vertex d, $z_{\mu(d)}$ is the amount of the (fully reduced) result vector that is completed at d and subsequently broadcast from it. The ReduceScatter sub-formulation compiles the reduced result to vertices in the amounts indicated by $z_{\mu(d)}$, and the AllGather sub-formulation distributes each portion to all vertices; the constraint $\sum_{d\in V_d} z_d = |V_d|$ forces the owned portions to account for exactly one full copy of the reduced vector. The formulation is free to choose any such z, which addresses the concern (§5.2) that a symmetric intermediate state (each vertex owning a $1/|V_d|$ share) need not be optimal on a non-symmetric topology.

We now argue that LP(20) is optimal. We proceed in two steps. First, we establish that restricting attention to algorithms that perform all reductions before all copies is without loss of optimality. Second, we show that, given this structure, the formulation computes an optimal routing solution. Throughout, we prove optimality at the level of the fluid (average-bandwidth) routing solution; converting to a concrete chunk-based schedule proceeds exactly as for a single collective (§4.3), by concatenating the ReduceScatter and AllGather schedules at the chunk level as described in §5.2.

Reduce-before-copy is without loss of optimality.

We first prove Theorem 5, which we restate here.

Theorem 5. *For the primary objective of collective completion time, the following holds: given an optimal AllReduce algorithm in which any piece of data is copied and later reduced, we can find an optimal algorithm with identical completion time where all pieces of data are first reduced and copied later.*

Rather than transforming a given algorithm step by step, we prove the theorem by extracting a *certificate* from an arbitrary algorithm that shows what rate of communication between pairs of GPUs the topology can support. We then show that this same certificate both lower bounds the completion time of the given algorithm and upper bounds the completion time achievable by algorithms that reduce before they copy.

For simplicity, we prove the theorem for the setting in which only GPUs are capable of copies and reductions, and where all links are symmetric. Writing $V_g = V_o \cup V_d$ for the set of GPUs participating in the collective and $n = |V_g|$, we assume that $V_c = V_r = V_g$, and that every remaining component may only store and transmit; we call these remaining vertices *relays*.

Call an algorithm *reduce-before-copy* if, for every piece of data, all reductions on it precede all copies of it. Equivalently, no partially reduced value is ever copied: for each element, the fully reduced result first comes into existence at a single GPU, and every transmission of that element thereafter is pure dissemination.

Because relays can neither copy nor reduce, data that enters a relay must leave it along a single outgoing edge. We may therefore view an algorithm as a collection of direct transmissions between two GPUs. Let

$$\mathcal{Y} = \Big\{ y \in \mathbb{R}_{\geq 0}^{\binom{n}{2}} : y \text{ is simultaneously routable in } G \text{ bidirectionally in one unit of time} \Big\},$$

that is, the set of rate matrices y indexed by unordered pairs of GPUs for which there is a multicommodity flow in G supplying rate y_{uv} from u to v *and* rate y_{uv} from v to u, for all pairs simultaneously, while respecting the bandwidth of every edge.

For $y \in \mathcal{Y}$ and a partition P of V_g into k_P parts, we write $y(\delta(P))$ for the total y-weight of the pairs of GPUs separated by P, and let

$$\rho(y) = \min_P \frac{y(\delta(P))}{k_P - 1}.$$

By the Tutte–Nash-Williams theorem [16, 22], $\rho(y)$ is exactly the fractional spanning tree packing number of the complete graph on V_g with edge capacities y.

Lemma 12. *Let $\mathcal{A}$ be any feasible AllReduce algorithm with collective completion time R, and let F_{uv} be the total volume of data delivered by transmissions of $\mathcal{A}$ from GPU u to GPU v. Then $\hat{y}_{uv} = \frac{1}{2R}(F_{uv} + F_{vu})$ lies in $\mathcal{Y}$.*

Proof. Aggregate the routes of all transmissions of $\mathcal{A}$, discarding self-loops and shortcutting any repeated vertex (which only decreases the load on every edge). The result is a path flow f in G in which each transmission from u to v contributes its volume along a u–v path. Since $\mathcal{A}$ respects the bandwidth of every edge at every instant, the volume that f places on a directed edge e is at most $b(e)R$; hence f/R is a feasible flow realizing the rate matrix F/R. Let f^{rev} be f/R with every path reversed. By symmetry, f^{rev} is also feasible, and it realizes the transpose of F/R. Finally, $g = \frac{1}{2}(f/R) + \frac{1}{2}f^{\text{rev}}$ places load at most $\frac{1}{2}b((u,v)) + \frac{1}{2}b((v,u)) = b((u,v))$ on each directed edge (u,v), and supplies rate $\frac{1}{2R}(F_{uv} + F_{vu}) = \hat{y}_{uv}$ in each direction for every pair of GPUs. □

Lemma 13. *Let P be any partition of V_g into k_P parts. Then every feasible AllReduce algorithm sends, for each element of data, at least $2(k_P - 1)$ transmissions that cross P (that is, whose two endpoints lie in different parts).*

Proof. Contract each part of P to a single supernode, and relax computation and communication within a supernode to be free and instantaneous. The crossing transmissions of the original algorithm are one-to-one with the transmissions of the contracted one, so it suffices to prove the bound after contraction.

Fix an element, let $t^\star$ be the first time at which it is fully reduced anywhere, say at supernode Q, and let ω be the result of that reduction. Let A be the set of transmissions that produced ω.

First, the transmissions to create ω contain exactly one occurrence of each of the n input sources. Consider a part $P_i \neq Q$ and any GPU $j \in P_i$: the occurrence of j's input in the lineage of ω must at some point leave P_i, via a transmission $m_i \in A$ whose sender lies in P_i. Transmissions with distinct senders are distinct, so the m_i are $k_P - 1$ distinct elements of A.

Next, every part $P_i \neq Q$ contains a GPU, and every GPU must eventually hold the fully reduced value; let $t_i \geq t^\star$ be the first time at which it is present in P_i. These times can only be the arrival time of some transmission m_i' received by P_i. If $t_i > t^\star$, then $m_i' \notin A$, because every transmission of A is received no later than $t^\star$. If $t_i = t^\star$, observe that any transmission of A that arrives exactly at $t^\star$ must be received by Q, so again $m_i' \notin A$. Thus, the m_i' are $k_P - 1$ distinct transmissions outside of A.

Together, at least $2(k_P - 1)$ transmissions cross P. □

Lemma 14. *For every $y \in \mathcal{Y}$, there is a reduce-before-copy AllReduce algorithm that does not exceed the rates given by y.*

Proof. By the Tutte–Nash-Williams theorem, pack spanning trees $\mathcal{T}_1, \ldots, \mathcal{T}_m$ of the complete graph on V_g with nonnegative weights $x_1, \ldots, x_m$ such that $\sum_{i:(u,v)\in\mathcal{T}_i} x_i \leq y_{uv}$ for every pair of GPUs and $\sum_i x_i = \rho(y)$, and split the data across the trees in proportion to the x_i. Every vertex of every $\mathcal{T}_i$ is a GPU, so we may root $\mathcal{T}_i$ at an arbitrary $r_i \in V_g$ and use it as follows: reduce the share of the data assigned to $\mathcal{T}_i$ upward to r_i, then broadcast the completed result downward. Each tree edge carries rate x_i from child to parent during the first phase and rate x_i from parent to child during the second. Hence the demand placed on a pair of GPUs (u,v) is at most y_{uv} in each direction, and since $y \in \mathcal{Y}$, all of these demands are simultaneously routable through G within its bandwidth limits. Each piece of data is assigned to a single tree, is fully reduced at that tree's root before it is ever copied, and is only copied thereafter; the algorithm is therefore reduce-before-copy. □

Proof of Theorem 5. Let $\mathcal{A}$ be any feasible AllReduce algorithm with collective completion time R, and let $\hat{y} \in \mathcal{Y}$ be the rate matrix supplied by Lemma 12. Fix a partition P of V_g. Summing Lemma 13 over input data size D, the total volume that crosses P in $\mathcal{A}$, counted in both directions, is at least $2D(k_P - 1)$; that is, $\hat{y}(\delta(P)) \geq \frac{D(k_P-1)}{R}$.

As P was arbitrary, $\rho(\hat{y}) \geq D/R$. Applying Lemma 14 to $\hat{y}$ produces a reduce-before-copy algorithm with collective completion time at most $D/\rho(\hat{y}) \leq R$. Taking $\mathcal{A}$ to be an optimal algorithm, the resulting reduce-before-copy algorithm is optimal as well. □

Optimality of the formulation.

By Theorem 5 we may restrict attention to reduce-before-copy algorithms. Such an algorithm splits, for each portion of the reduced vector, into a *reduce phase* that carries partial sums up to the vertex where that portion is completed, and a *copy phase* that broadcasts the completed portion outward. We formalize the correspondence to the two sub-formulations.

Fix the intermediate state z. The ReduceScatter sub-formulation $(f^{(1)}, w^{(1)})$ is then precisely LP(9) for the ReduceScatter collective with per-destination demand $z_{\mu(d)}$ (on the reversed graph, with copy vertices V_c replaced by reduce vertices V_r, as in §5.2); and the AllGather sub-formulation $(f^{(2)}, w^{(2)})$ is precisely LP(9) for the AllGather collective with per-origin supply $z_{\mu(o)}$. This lets us apply Theorem 2 to each phase.

Lemma 15. *Fix z. The optimal value of the ReduceScatter sub-formulation equals the optimal value of the corresponding step-based formulation LP(7), and likewise for the AllGather sub-formulation.*

Proof. Theorem 2 states that LP(9) and LP(7) are equivalent for AllGather. ReduceScatter is AllGather on the reversed graph with V_c replaced by V_r, so the same theorem applies verbatim. The only coupling between the two sub-formulations is (i) the shared intermediate state z, which is held fixed here, and (ii) the shared bandwidth constraint. For (ii), the staggering construction in the proof of Theorem 2 (§A.3) divides the data into K chunks and staggers their transmissions so that an average-bandwidth constraint becomes a per-step constraint; applying the same construction to the concatenated ReduceScatter-then-AllGather with a common K converts the shared average constraint $\sum_{t_1} \cdots + \sum_{t_2} \cdots \le Rb(L)$ into a per-step bound $s'b(L)$, while the chunk-level concatenation (a chunk enters AllGather only once its ReduceScatter completes) preserves the reduce-before-copy ordering for every chunk. □

As with a single AllGather (§4.2), an LP(9) solution for either phase may fail to be algorithm-feasible on a non-ideal topology, because a usage variable underestimates the true traffic when disjoint pieces of data from a common origin share a link. The lazy vertex-duplication strengthening of §4.2 applies to each sub-formulation independently: we duplicate offending vertices in $G^{T^{(1)}}$ and $G^{T^{(2)}}$ separately and re-solve, until both phases are algorithm-feasible. Since every relaxation is a lower bound and the two phases interact only through the (linear) shared bandwidth constraint and the shared z, the strengthened formulation remains a valid relaxation, so its optimum is still a lower bound and, once feasible, is optimal.

Theorem 16. *LP(20) computes an optimal AllReduce routing solution: for any target collective completion time R, it is feasible if and only if there is a reduce-before-copy AllReduce routing solution with collective completion time at most R.*

Proof. Given an optimal solution to LP(20), Lemma 15 (with lazy duplication where needed) makes each phase an optimal, algorithm-feasible fluid routing solution. Concatenating them yields a valid reduce-before-copy AllReduce whose average per-edge traffic over the combined lifetime R is the sum of the two phases' traffic, so the shared bandwidth constraint is exactly the AllReduce bandwidth budget; the collective completion time is at most R and no algorithm can have a lower completion time. □

A.10 Additional details from §6

First, we more formally define indistinguishability. We say that two GPUs g_1 and g_2 are indistinguishable if there is a bijection $\chi : V \to V$ such that $\chi(g_1) = g_2$ and for all vertices $v_1, v_2 \in V$,

- $(v_1, v_2) \in E$ if and only if $(\chi(v_1), \chi(v_2)) \in E$;
- $v \in V_o$ if and only if $\chi(v) \in V_o$ (and analogously for V_b, V_c, V_r, V_d);
- $c_{(v_1,v_2)} = c_{(\chi(v_1),\chi(v_2))}$ for $(v_1, v_2) \in E$;
- $b(L) = b(\{(\chi(v_1), \chi(v_2)) : (v_1, v_2) \in L\})$ for $L \in \mathcal{L}$.

Now, recall that in the main text, we defined a Symmetry Partition $\mathcal{P}$ as a partition of V_o into "indistinguishability classes" where for any $P \in \mathcal{P}$ and any $g_1, g_2 \in P$, g_1 and g_2 are indistinguishable.

Next we prove a useful lemma about Symmetry Partitions:

Lemma 17. *Given a Symmetry Partition $\mathcal{P}$, there are functions $\pi : V \times [|P|] \to V$ for each $P \in \mathcal{P}$ such that*

- *$\pi(\cdot, i)$ is a certificate of indistinguishability for v, $\pi(v, i)$ for $v \in P$ for all i,*
- *$\{\pi(v, i) : i \in [|P|]\} = P$ for all $v \in P$ and $\{\pi(v, i) : v \in P\} = P$ for all i.*

This lemma says that after partitioning V_o into sets of indistinguishable vertices, for each set, we can find a series of relabelings of G that preserve all properties of the graph. These relabelings are all bijections on our set of indistinguishable vertices, and they verify the indistinguishability of all pairs of vertices in this set.

Proof. We let H be the group of all bijections $\chi : V \to V$ that preserve the graph structure (that is, they satisfy the above four conditions). Then we may say that two vertices are indistinguishable if and only if they are in the same orbit under the H-action. And therefore, the parts in our Symmetry Partition $\mathcal{P}$ exactly correspond to orbits of V_o.

Now we construct π for each $P \in \mathcal{P}$. Fix some $v_0 \in P$. Then we know $P = \{h_1(v_0), \ldots, h_{|P|}(v_0)\}$ for some $h_1, \ldots, h_{|P|} \in H$. We let $\pi(v, i) = h_i(v)$.

First, we note that by the definition of our group action, $\pi(\cdot, i)$ acts as a certificate of indistinguishability. And since each h_i is a bijection,

$$\{\pi(v, i) : v \in P\} = \{h_i(v) : v \in P\} = P.$$

Finally, for any $v \in P$, there is some $h' \in H$ such that $h'(v_0) = v$, so

$$\begin{aligned}\{\pi(v, i) : i \in [|P|]\} &= \{h_i(v) : i \in [|P|]\} \\ &= \{h_i(h'(v_0)) : i \in [|P|]\} \\ &= \{h_i(v_0) : i \in [|P|]\} = P.\end{aligned}$$

Therefore, our constructed π satisfies the requirements in the lemma. □

In the following, we will extend the functions π in the natural way to also apply to edges: $\pi((v_1, v_2), i) = (\pi(v_1, i), \pi(v_2, i))$ and sets: $\pi(W, i) = \{\pi(v, i) : v \in W\}$. We further extend the notation to also work on vertices and edges of our constructed time-expanded networks: $\pi(v^T, i) = (\pi(\mu(v), i), \tau(v^T))$ for $v^T \in V^T$, and $\pi((v_1^T, v_2^T), i) = (\pi(v_1^T, i), \pi(v_2^T, i))$ for $(v_1^T, v_2^T) \in E^T$.

Now, we establish that formulation LP(9) (and other similar LP formulations) has optimal symmetric solutions.

A solution is symmetric with respect to π if for all $i \in [|P|]$, $f_{\pi(e,i),\pi(o,i),\pi(d,i)} = f_{e,o,d}$ for all f variables and $w_{\pi(e,i),\pi(o,i)} = w_{e,o}$ for all w variables.

Now we give a more precise statement of the theorem before proving it.

Theorem 6. *Given a solution to LP(9), Symmetry Partition $\mathcal{P}$, and associated functions π for each $P \in \mathcal{P}$, we can find a solution to LP(9) of the same completion time and objective value that is symmetric with respect to every π.*

Proof sketch. Take any solution f^0, w^0 to LP(9). We will describe a process to find a symmetric solution with equal cost. We describe the argument for f, but the same argument holds for w.

Table 3: Algorithm generation runtime (s) of standard Dantzig-Wolfe vs. Mirrored Dantzig-Wolfe. Missing entries did not finish to optimality within a 10-hour time limit.

GPUs	Standard Dantzig-Wolfe	Mirrored Dantzig-Wolfe
8	60.5	2.95
16	2610	27.6
32	34700	82.2
64	—	634
128	—	9870

Consider the sets $P_1, \ldots, P_r \in \mathcal{P}$ and their corresponding functions $\pi_1, \ldots, \pi_r$. For each choice of j from 1 to r, we compute $f^j_{e,o,d} = \sum_{i \in [|P_j|]} f^{j-1}_{\pi_j(e,i),\pi_j(o,i),\pi_j(d,i)} / |P_j|$.

Then we can check that f^j is symmetric with respect to π_j. We can also check that if f^{j-1} is symmetric w.r.t. some $\pi_{j'}$, then f^j is also symmetric w.r.t. $\pi_{j'}$. These two statements are enough to conclude that f^r is symmetric with respect to every π. □

Now we can constrain our solution to ensure that

$$f_{\pi(e,i),\pi(o,i),\pi(d,i)} = f_{e,o,d} \quad \text{for } i \in [|P|], P \in \mathcal{P}$$

(and the same for w variables). And as mentioned in the main text, this allows us to consider just one subproblem per indistinguishability class.

To be concrete, given Symmetry Partition $\mathcal{P}$, for each $P_k \in \mathcal{P}$, choose $o_k = (v, 0) \in V^T_o$ for some $v \in P_k$. Then we create subproblems (S_k) for each indistinguishability class P_k:

$$\begin{aligned}
\min_{f,w} \quad & \sum_{e \in E} \sum_{t=0}^{T-1} \hat{c}_e w_{\kappa(e,t),o_k} \\
\text{s.t.} \quad & \sum_{e \in \delta^-(v)} f_{e,o_k,d} - \sum_{e \in \delta^+(v)} f_{e,o_k,d} = 0 \\
& \qquad\qquad \text{for } d \in V^T_d, v \in V^T \setminus \{o_k, d\} \\
& \sum_{e \in \delta^-(o_k)} f_{e,o_k,d} - \sum_{e \in \delta^+(o_k)} f_{e,o_k,d} = 1 \qquad \text{for } d \in V^T_d \\
& \sum_{e \in \delta^-(d)} f_{e,o_k,d} - \sum_{e \in \delta^+(d)} f_{e,o_k,d} = -1 \qquad \text{for } d \in V^T_d \\
& w_{e,o_k} \geq f_{e,o_k,d} \qquad \text{for } e \in E^T, d \in V^T_d \\
& f, w \geq 0.
\end{aligned}$$

In our master problem, we replace the bandwidth constraint with the constraint

$$\sum_{t=0}^{T-1} \sum_{e \in L} \sum_{P_k \in \mathcal{P}} \sum_{j \in J_{o_k}} \sum_{i=1}^{|P_k|} \lambda^j_{o_k} \bar{w}^j_{\kappa(\pi_k(e,i),t),o_k} \leq Rb(L)$$

for each bandwidth constraint $L \in \mathcal{L}$, where π_k is the associated function for indistinguishability class P_k.

A.11 Benefits of exploiting symmetry in OptCCL

To isolate the contribution of our novel Mirrored Dantzig-Wolfe technique, we compare our approach against a standard Dantzig-Wolfe implementation. This analysis shows that our symmetry-exploiting modifications are not merely beneficial but essential for achieving computational tractability at scale. Table 3 shows the generation time comparison for AllGather between standard Dantzig-Wolfe decomposition and our Mirrored Dantzig-Wolfe approach across increasing cluster sizes. We note that the relative performance gain increases with the size of the cluster.

A.12 Extended discussion of SyCCL

SyCCL defines symmetry through a hierarchical dimensional structure: links are classified into dimensions based on connection type (*e.g.*, NVLinks form one dimension, NIC-to-rail links are another), and GPUs are partitioned into groups within each dimension based on connectivity (*e.g.*, GPUs connected to the same rail form a group). Groups within the same dimension are isomorphic with respect to the associated links.

Based on this structure, SyCCL [5, §3.1] claims without proof that "optimal schedules consist of consistent sub-schedules across isomorphic GPU groups", that is, the communication pattern among GPUs in one group should be identical to that in any isomorphic group, ignoring cross-dimension communication. (Our counterexample to this claim can be found below.) SyCCL then generates "sketches" that decompose collectives into stages of sub-demands, and the prior claim is used to prune sketches that have different sub-demands for different isomorphic groups.

Even ignoring the above concern, SyCCL's techniques introduce several further sources of potential suboptimality. First, SyCCL acknowledges that their method of finding solutions for sub-demands may be suboptimal because of their MILP modeling strategy. Second, SyCCL uses other heuristics to prune sketches, which may also discard optimal solutions. Third, each sub-demand is solved independently, but optimal sub-solutions do not necessarily compose into an optimal global solution, since interference on shared links between sub-demands is ignored during optimization.

Optimality without symmetry.

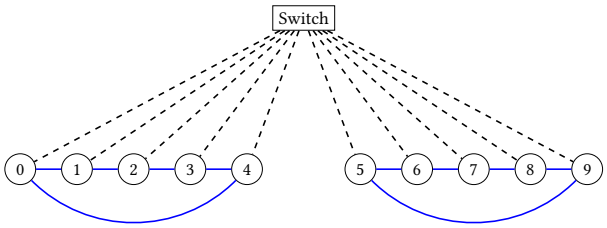


Figure 6: A topology to demonstrate no symmetric solution is optimal.

We provide an example in which an optimal schedule is not consistent across isomorphic groups of GPUs and, furthermore, in which no symmetric solution is optimal.

Consider the topology depicted in Figure 6, where we define the solid blue links to be twice as fast as the dashed links to the switch. We focus on finding a Broadcast algorithm for a single indivisible chunk of data. In this setting, we can assume that the data requires

one time step to travel along a fast link, and two time steps to travel through the switch.

We first describe a Broadcast algorithm that completes in three time steps:

- Step 1: We transmit data from GPU 0 to GPU 1 and GPU 4 along fast links, and to GPU 5 through the switch.
- Step 2: We transmit data from GPU 1 to GPU 2 and from GPU 4 to GPU 3 along fast links. We also transmit data from GPU 1 to GPU 7 and GPU 4 to GPU 8 through the switch.
- Step 3: Now that data has arrived at GPU 5, we transmit data from GPU 5 to GPU 6 and to GPU 9 via fast links.

At the end of step 3, all ten GPUs have received the data.

We now argue that no solution with consistent sub-schedules on the two isomorphic sets of vertices ($\{0, 1, 2, 3, 4\}$ and $\{5, 6, 7, 8, 9\}$) can finish in three steps.

First, the solution above is not symmetric in this way: it uses four links between the vertices in $\{0, 1, 2, 3, 4\}$, while using only two between the vertices in $\{5, 6, 7, 8, 9\}$.

Second, if we transmit only one chunk through the switch, the earliest that data can arrive at all destinations is after four time steps. After three time steps, only three GPUs among $\{5, 6, 7, 8, 9\}$ can have received the data.

Thus, for any solution to finish in three time steps, it must send data through the switch to the group of GPUs $\{5, 6, 7, 8, 9\}$ at least three times. (Two is not sufficient because only one chunk may be transmitted at time step 1, so subsequent transmissions will arrive no sooner than the end of time step 3.)

Since we are searching for a symmetric solution, this means we can use at most two of the fast links among GPUs $\{0, 1, 2, 3, 4\}$. So for the data to reach all destinations in such a symmetric solution, we require five transmissions through the switch, which cannot be completed in three time steps. We therefore conclude that no solution with identical communication patterns across the two isomorphic GPU groups can finish in three time steps.

This example shows that SyCCL's assumption that isomorphic sub-networks employ identical communication patterns can preclude optimal solutions.

A.13 Comparison with concurrent work, and illustrative example

As we noted in Table 1, ForestColl is limited to a subset of the collectives we consider as well as a subset of hardware features. Like other spanning tree packing methods, it does not produce algorithms for All-to-All, and is unable to handle memory vertices. Here, we demonstrate that for the limited scenarios where ForestColl can generate algorithms, it is scalable and can produce optimal algorithms. To run ForestColl on these topologies, we treat memory components as switches, and we impose the memory bandwidth constraint individually on every link incident to memory. In Figure 7, we compare the AllGather scalability and performance of OptCCL against ForestColl on the A100 topology without NVLinks and on the DOE topology without NVLinks. In the first scenario, ForestColl can synthesize an algorithm on 128 GPUs in 20 seconds compared to just over one hour for OptCCL, and achieves the same optimal algorithmic bandwidth as OptCCL.

However, ForestColl does not always produce optimal algorithms. In the DOE case, while ForestColl produces algorithms in under 30 seconds compared to under 300 seconds for OptCCL, it cannot achieve the same algorithmic bandwidth. In fact, at all topology sizes, ForestColl performs about 40% worse. This discrepancy in performance stems from the shared, half-duplex nature of memory bandwidth: since ForestColl is unable to encode the half-duplex memory bandwidth constraint, it produces algorithms that overuse memory, leading to much lower algorithmic bandwidth. We elaborate on this with examples below.

Examples. We now explore existing methods on several toy examples, illustrating various ways in which synthesizing collective communication algorithms is challenging and highlighting some drawbacks of existing works.

Step-based algorithms. We begin with an example showing that step schedules always lose some performance in comparison to continuous routing solutions. This is not surprising, but we make the point concrete. (ForestColl makes this argument more robustly.)

In any step schedule, we fix a minimum atomic unit of data (usually called a chunk). This quantization of data prevents us from achieving the same completion time as if we could transfer infinitely small units of data. Consider the Broadcast collective on the topology in Figure 8. In an optimal routing solution, both the link from 1 to 2 and the link from 2 to 3 should be fully saturated at all times. However, in any step solution, when the last chunk is transmitted from 2 to 3, the link from 1 to 2 is unused. So no step solution can achieve the same completion time as a routing solution.

Modeling memory. No existing method provides the ability to explicitly model memory as a component in its representation of topologies. Explicitly modeling memory is useful because it enables algorithms that could not previously be considered, as our earlier example shows.

Consider the topology in Figure 9. In a Broadcast from 1, without modeling memory, one optimal algorithm transmits data from GPU 1 to GPU 2 and GPU 2 to GPU 3. However, given the real structure, what this algorithm actually entails is writing data from GPU 1 to memory and then reading that data to GPU 2. We then write the data from GPU 2 to memory, and GPU 3 reads that data. If we correctly model memory, then we can find an algorithm that writes the data from GPU 1 to memory, then both GPU 2 and GPU 3 read that data.

In a model with discrete chunks of data and uniform link bandwidths, the first algorithm requires four steps, where one step is the duration of a single transmission. Our second algorithm requires only two steps, because both reads from memory can be done in parallel. Even if we do not use a model with discrete data chunks, our second algorithm requires writing less data to memory.

Where packing spanning trees fails. We now show the ways in which spanning tree packing methods struggle to properly model data transmissions, particularly with respect to memory. We note that these examples focus on Broadcast, whereas ForestColl does not claim to generate algorithms for Broadcast. It does generate algorithms for AllGather, and we use these simpler Broadcast examples to show ways in which the algorithms it generates for AllGather may be suboptimal.

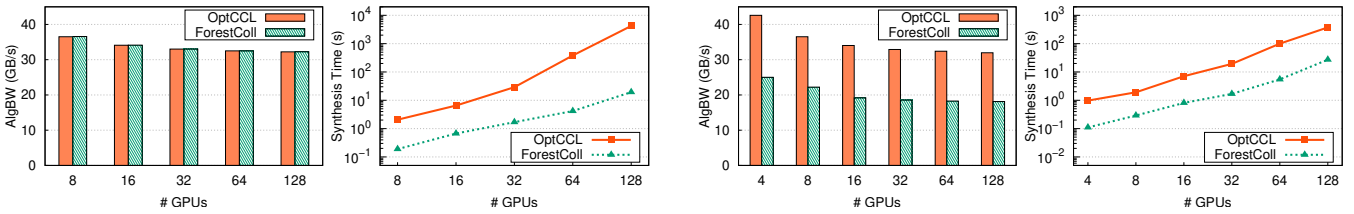


Figure 7: Performance in terms of algorithmic bandwidth and synthesis time for the A100 (left) and the DOE topologies (right) without NVLinks for the AllGather collective. OptCCL synthesizes algorithms with performance equivalent to ForestColl or up to 1.4× better, but requires 10–200× longer synthesis time.

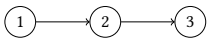


Figure 8: A topology to demonstrate pipelining.

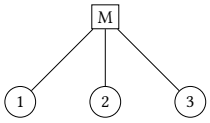


Figure 9: A topology to demonstrate using memory.

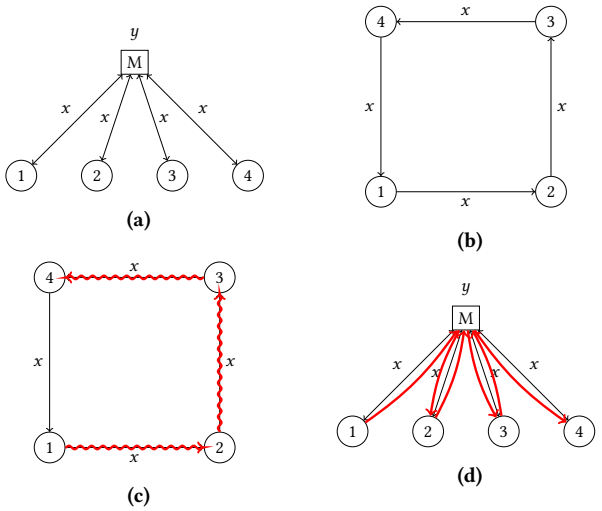


Figure 10: A topology to show the difficulty of modeling memory bandwidth.

Shared bandwidth. We begin by showing how ForestColl and other spanning tree packing methods are unable to handle more complicated bandwidth limitations. In addition to link bandwidths, we may also want to consider "component bandwidths". In our examples, we consider memory bandwidth: a limit on the amount of data that can be read from and written to the host memory.

Consider a simple topology like the one in Figure 10a, where we have four GPUs all connected to a host memory. Let x be the bandwidth on each of the links, and y be the memory bandwidth. For the purpose of this example, suppose that our only goal is to treat the memory as a switch, except that it has its own bandwidth limitation. (This means we are ignoring the fact that memory components typically allow storing and copying data.) Then in ForestColl's implementation, we would perform a process called vertex splitting to replace the switch with some subset of the implied direct edges connecting the incident vertices. The result of this vertex splitting produces a graph like Figure 10b, where the vertices incident to the switch now form a ring. Then some optimal solution on that topology for Broadcast might look like Figure 10c, where the highlighted red edges indicate how data should travel in a steady-state. And when we map this solution back to the original topology, we get an optimal spanning tree as described in Figure 10d. But this entirely ignores the memory bandwidth: if $y < 6x$, then the algorithm described in Figure 10d would not have the expected throughput of x, but rather a throughput of $y/6$, and the solution it finds would not be optimal.

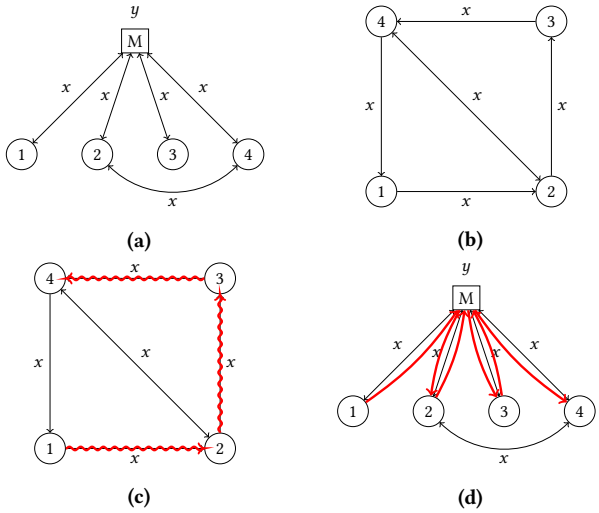


Figure 11: An unnatural topology to emphasize the difficulty of modeling memory bandwidth.

We can see an even more extreme version of this phenomenon by considering a more unnatural topology like the one in Figure 11a. Though such a topology is unlikely to exist in practice, this example emphasizes the need for proper handling of memory vertices. In this example, the same basic logic holds as before: Vertex splitting would yield a representation like the one in Figure 11b. Figure 11c highlights one optimal solution on that representation, and Figure 11d shows the mapping back to the original topology. Not only is the solution suboptimal in this case; there is a better solution that routes data along a different path, using edges $(1, 2)$, $(2, 3)$, $(2, 4)$

and sending x units of data along each one. Since this proposed solution only uses 4 edges incident to the memory component, it would have throughput equal to $\min\{y/4, x\}$. Thus, beyond the problem of reporting the correct optimal throughput, the path along which data is transmitted may also differ in an optimal solution.

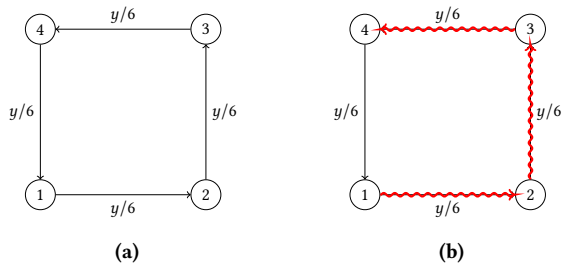


Figure 12: An attempt to adjust link bandwidths.

One possible remedy is to adjust the link bandwidths when we remove the switch so that they reflect the memory bandwidth. Indeed, setting the link bandwidths from the example beginning in Figure 10 to $\min\{y/6, x\}$, as in Figure 12a, does work in this simple example. But we cannot guarantee that such an approach will work in all cases.

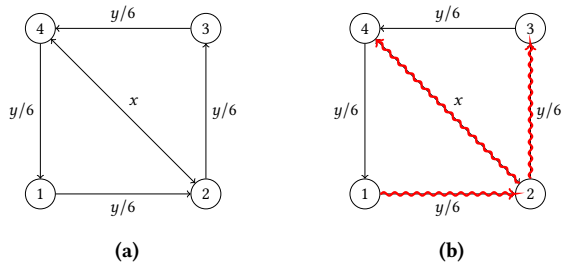


Figure 13: Problems with adjusting link bandwidths.

In more complex topologies, where the data can be routed along an alternative path that avoids the depicted memory vertex (as in Figure 11), even this strategy yields a suboptimal solution. Applying the same adjustment in this case gives the bandwidths shown in Figure 13a. An optimal spanning tree is in Figure 13b, and the tree in Figure 12b also has the same apparent throughput. But if we map Figure 13b back to a tree on the original graph, we can observe that this tree supports a throughput of $\min\{y/4, x\}$. Once again, the solution returned is not optimal. (And since we may return Figure 12b instead of Figure 13b, the data may even be transmitted differently from the optimal algorithm.)

A final alternative is to separate the memory vertex in our representation into two vertices joined by a one-directional link with the desired memory bandwidth. This correctly models our situation, but ForestColl is unable to handle this, since it requires that input graphs be Eulerian, which this modified input is not.

Memory as a non-mandatory compute vertex. Memory behaves in a way that is distinct from other components. It allows data to be copied, like a GPU, but it typically cannot perform reductions. And unlike GPUs, it is not necessary to even use memory in a solution.

While it is not hard to imagine supporting this behavior within a linear programming formulation such as those of TE-CCL or TACCL, we give an example showing that this concept is fundamentally incompatible with spanning tree packing.

The technique used by ForestColl finds spanning trees on its graph representation. Two requirements are at odds here: to properly model memory, we want to treat it as a vertex in the graph so that it can copy data; but data need not be transmitted to the corresponding vertex, so the objects we seek are no longer spanning trees, but Steiner trees. Packing Steiner trees, however, is NP-hard [6].

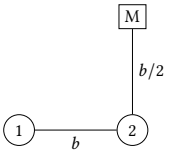


Figure 14: A topology to demonstrate treating memory as a GPU fails.

Thus, to use spanning tree packing, we must either give up copying at memory (treating it as a switch) or require data to be transmitted to all memory (treating it as a GPU). We have already described above the obstacles to accurately modeling memory as a switch. Figure 14 shows a simple example where treating memory as a GPU produces worse solutions. For a Broadcast from vertex 1, treating memory as a GPU produces a solution tree that uses both links, with throughput $b/2$; the optimal solution uses only the link connecting vertices 1 and 2, achieving a throughput of b.